

Apstrakt

U ovom diplomskom radu predstavljeni su projektovanje, implementacija i verifikacija platforme LocalBite, namenjene praćenju proizvodnje, zaliha, prodaje i porekla domaćih proizvoda. Rad razmatra problem nepovezane evidencije sirovina i proizvodnje na malim proizvodnim preduzećima, kao i ograničenu dostupnost informacija o poreklu proizvoda u trenutku kupovine. Predloženo rešenje povezuje podatke o korisnicima, preduzećima, sirovinama, proizvodnim serijama, gotovim proizvodima i porudžbinama u jedinstven tok, čime omogućava sledljivost od ulaznih sirovina do javnog prikaza porekla dostupnog putem QR koda.

Sistem je realizovan kao mikroservisna aplikacija koju čini pet poslovnih servisa i servis za formiranje i čitanje projekcija podataka, pri čemu svaki servis koristi zasebnu PostgreSQL bazu podataka. Komunikacija između servisa zasniva se na REST pozivima i asinhronoj razmeni događaja posredstvom sistema RabbitMQ. Posebna pažnja posvećena je upravljanju transakcijama, idempotentnosti operacija, izolaciji podataka po preduzeću, atomskoj rezervaciji zaliha i oporavku nedovršenih distribuiranih poslovnih tokova.

Ključne reči: mikroservisi, proizvodna preduzeća, sledljivost, REST, RabbitMQ, PostgreSQL, CQRS, transakcioni outbox, QR kod

Sadržaj

1. 1 Uvod
2. 2 Problem i cilj sistema
3. 3 Analiza domena i funkcionalni zahtevi
4. 4 Teorijske osnove i izbor tehnologija
5. 5 Arhitektura sistema i primenjeni softverski obrasci
6. 6 Model podataka i servisne baze
7. 7 Implementacija backend servisa
8. 8 Integracija servisa i distribuirani tokovi
9. 9 Frontend i korisnički interfejs
10. 10 Bezbednost, pouzdanost i ograničenja
11. 11 Testiranje i verifikacija
12. 12 Pokretanje, održavanje i oporavak
13. 13 Diskusija rezultata i budući razvoj
14. 14 Zaključak

Literatura

Prilog A API primeri

Prilog B Statusi i matrica prava

Spisak slika

Slika 1 Kontekst sistema

Slika 2 Servisne granice i tok podataka

Slika 3 Unutrašnja slojevita organizacija servisa

Slika 4 Tok proizvodnje i nastanka zalihe

Slika 5 Tok porudžbine i rezervacije

Slika 6 Tok javnog prikaza porekla

Slika 7 Tok transakcionog outbox-a

Slika 8 Model statusa proizvodne serije

Slika 9 Model statusa porudžbine

Spisak tabela

Tabela 1 Interesne grupe i ciljevi

Tabela 2 Katalog funkcionalnih zahteva

Tabela 3 Uloge i dozvole

Tabela 4 Servisi i vlasništvo nad podacima

Tabela 5 Pregled tehnologija

Tabela 6 Matrica kvarova

Tabela 7 Rezultati verifikacije

Tabela 8 Tehnički dug i naredne aktivnosti

1 Uvod

1.1 Motivacija

Mala proizvodna preduzeća mogu voditi evidenciju o sirovinama, proizvodnim postupcima, zalihama i prodaji u sveskama, elektronskim tabelama i drugim međusobno nepovezanim izvorima. Ovakva organizacija podataka otežava utvrđivanje raspoloživih količina, planiranje proizvodnje i naknadno praćenje porekla gotovih proizvoda. Ograničenje takvog pristupa ne proizlazi isključivo iz ručnog unosa podataka, već i iz odsustva veza između zapisa o pojedinačnim fazama proizvodnje i prodaje.

Značaj povezivanja ovih podataka može se ilustrovati hipotetičkim primerom proizvodnje ajvara. U prvoj proizvodnoj seriji, 1. septembra 2026. godine, proizvedeno je 30 kg ajvara, dok je 3. septembra proizvedeno dodatnih 50 kg od paprika iz druge isporuke. Ukoliko se naknadno utvrdi neispravnost paprika upotrebljenih u prvoj seriji, potrebno je identifikovati proizvode nastale njihovom preradom. Bez evidencije koja povezuje isporuke sirovina sa proizvodnim serijama, moglo bi biti neophodno povući svih 80 kg proizvoda zbog nemogućnosti pouzdanog razgraničenja serija. Povezivanjem ovih zapisa omogućava se identifikacija potencijalno zahvaćenih 30 kg, dok QR kod olakšava pristup podacima o poreklu odgovarajućeg proizvoda. Mogućnost ograničavanja obuhvata povlačenja zavisi od potpunosti i tačnosti evidentiranih podataka.

Sa stanovišta kupca, dostupnost podataka o proizvođaču, proizvodnoj seriji, korišćenim sirovinama i redosledu proizvodnih koraka doprinosi transparentnosti proizvodnje i može podržati poverenje u domaći proizvod. QR kod predstavlja pristupnu tačku tim podacima. Platforma LocalBite stoga povezuje proizvod iz kataloga sa proizvodnom serijom i pripadajućim sirovinama, omogućavajući uvid u evidentirani tok njegovog nastanka.

1.2 Predmet i cilj rada

Predmet rada je razvoj mikroservisne platforme za praćenje proizvodnje i porekla domaćih proizvoda. Osnovni cilj rada je projektovanje i implementacija sistema koji podržava rad više proizvodnih preduzeća u okviru jedne aplikacije, uz kontrolisan pristup podacima svakog preduzeća. Sistem treba da omogući vlasnicima i radnicima vođenje operativne evidencije, kupcima pregled ponude i kreiranje porudžbina, a posetiocima pristup javnom prikazu porekla proizvoda skeniranjem QR koda, bez prethodne registracije.

Dodatni cilj je oblikovanje proširivog sistema primenom softverskih obrazaca i mikroservisne arhitekture. Takav pristup treba da olakša uključivanje novih preduzeća i podršku za različite tipove

preduzeća, sirovina, proizvoda i proizvodnih koraka. Poseban cilj odnosi se na razumljivost korisničkog interfejsa i jasnoću poslovnih postupaka, kako bi platformu mogli da koriste i vlasnici i radnici bez prethodnog iskustva sa sličnim informacionim sistemima.

1.3 Metod i struktura rada

Rad obuhvata analizu problema i zahteva, projektovanje arhitekture i modela podataka, implementaciju servisa i proveru njihovog ponašanja. Opis rešenja zasniva se na projektnoj dokumentaciji, izvornom kodu, definisanim HTTP interfejsima, testovima i skriptama za pokretanje, održavanje i oporavak sistema. Pri razmatranju rezultata razlikuju se realizovane funkcionalnosti, funkcionalnosti sa ograničenom podrškom i predlozi za budući razvoj. Organizacija narednih poglavlja prikazana je u tabeli 1.

Poglavlje	Sadržaj
2–3	Poslovni problem, učesnici, zahtevi i slučajevi korišćenja
4–5	Teorijske osnove, tehnologije, arhitektonske odluke i softverski obrasci
6–9	Model podataka, serverska aplikacija, integracija servisa i korisnički interfejs
10–12	Bezbednost, testiranje, pokretanje i oporavak
13–14	Diskusija rezultata, budući razvoj i zaključak

Tabela 1: Organizacija rada

2 Problem i cilj sistema

2.1 Poslovni problem

Osnovni poslovni problem predstavlja nepovezanost podataka o ulaznim sirovinama, proizvodnim serijama, gotovim proizvodima i prodajnim transakcijama. Kada se prijem sirovina, njihova potrošnja i prodaja proizvoda evidentiraju u odvojenim sistemima, otežano je usaglašavanje evidencije sa stvarnim stanjem zaliha. Istovremeno, odsustvo veza između ovih zapisa otežava rekonstrukciju proizvodnog toka i utvrđivanje porekla pojedinačnog proizvoda.

Dodatni problem odnosi se na upravljanje pristupom podacima u sistemu koji koristi više preduzeća. Interni poslovni podaci jednog preduzeća moraju biti zaštićeni od neovlašćenog pristupa korisnika drugih preduzeća, uz jasno razdvajanje od podataka namenjenih javnom prikazu. Identifikator preduzeća (farm_id) zato predstavlja jedan od elemenata kontrole pristupa. Pri čitanju ili izmeni zaštićenih poslovnih podataka potrebno je proveriti ovlašćenja korisnika i pripadnost podataka odgovarajućem preduzeću, u skladu sa pravilima za konkretnu korisničku ulogu.

2.2 Interesne grupe

Interesna grupa	Potrebe i očekivanja	Podrška sistema
Vlasnik preduzeća	Upravljanje proizvodnjom, zaposlenima i ponudom	Funkcionalnosti za upravljanje preduzećem, sirovinama, proizvodnjom i proizvodima
Radnik	Efikasno evidentiranje operativnih promena	Unos i izmena podataka o sirovinama i proizvodnim serijama u okviru dodeljenih ovlašćenja
Kupac	Pregled ponude i praćenje porudžbina	Katalog proizvoda, korpa i pregled porudžbina i njihovih statusa
Posetilac	Pristup informacijama o poreklu bez korisničkog naloga	Javni prikaz porekla putem QR koda i dokument o poreklu u PDF formatu
Administrator	Administrativni pregled i podrška korisnicima	Pristup administrativnim pregledima u okviru definisanih ovlašćenja

Tabela 2: Interesne grupe i ciljevi

2.3 Granice sistema

Obuhvat platforme LocalBite čine evidencija poslovnih podataka i podrška tokovima proizvodnje i prodaje unutar sistema. Platforma obuhvata registraciju i prijavu korisnika, upravljanje preduzećima i radnicima, evidenciju sirovina i proizvodnih serija, upravljanje proizvodima i njihovim slikama, generisanje QR kodova, prikaz porekla, izradu dokumenata o poreklu u PDF formatu, obradu porudžbina i

analitičke preglede. Elektronsko plaćanje, fiskalizacija, integracija sa službama dostave, povezivanje sa hardverskim senzorima i nezavisna sertifikacija podataka nisu obuhvaćeni rešenjem.

Pri tumačenju javnog prikaza porekla potrebno je razlikovati sledljivost evidentiranih podataka od nezavisne potvrde njihove tačnosti. Dokument o poreklu u PDF formatu zasniva se na podacima koje je proizvođač uneo u sistem. Povezanost zapisa omogućava praćenje evidentiranog proizvodnog toka, ali sama po sebi ne potvrđuje usklađenost unetih podataka sa stvarnim stanjem. Stoga ovaj dokument ne predstavlja nezavisni sertifikat kvaliteta ili ispravnosti proizvoda.

2.4 Ciljevi kvaliteta

- Izolacija internih podataka po preduzeću i kontrola pristupa prema korisničkoj ulozi i dodeljenim ovlašćenjima;
- očuvanje istorijskih vrednosti podataka o korišćenim sirovinama i cenama;
- idempotentnost operacija potrošnje sirovina i rezervacije zaliha, tako da ponavljanje istog zahteva ne izazove višestruku promenu stanja;
- otpornost projekcija podataka na ponovljene događaje i događaje pristigle van redosleda;
- mogućnost ponovne obrade događaja radi obnove projekcija nakon oporavka sistema;
- jasno razdvajanje merodavnog stanja zaliha od javnog modela za čitanje, koji se ažurira asinhrono.

3 Analiza domena i funkcionalni zahtevi

Analiza domena obuhvata učesnike u radu platforme, poslovne objekte i pravila koja određuju njihove međusobne odnose. U sistemu LocalBite osnovni poslovni tok započinje evidentiranjem sirovina, nastavlja se kroz proizvodnu seriju i njene korake, a završava nastankom proizvoda i obradom porudžbina. Podaci nastali u ovom toku koriste se i za javni prikaz porekla proizvoda. U ovom poglavlju funkcionalni zahtevi razmatraju se u skladu sa realizovanim servisima, proverama ulaznih podataka i podržanim promenama stanja.

3.1 Akteri i korisničke uloge

Platforma razlikuje pet kategorija krajnjih korisnika: posetioca, kupca, vlasnika preduzeća, radnika i administratora sistema. Njihova ovlašćenja određuju se prema korisničkoj ulozi i, kada je to primenljivo, pripadnosti preduzeću. Pripadnost preduzeću predstavlja osnov za razdvajanje internih poslovnih podataka više proizvođača koji koriste istu platformu.

Posetilac pristupa javnom prikazu porekla preko QR koda, bez registracije i prijave. Dostupni su mu podaci namenjeni javnom prikazu i dokument o poreklu u PDF formatu. Ovakav pristup ne podrazumeva ovlašćenje za pregled interne evidencije preduzeća, podataka o zaposlenima ili porudžbina drugih korisnika.

Kupac je registrovani korisnik sa ulogom CUSTOMER. Pregleda prodajnu ponudu, koristi dostupne filtere, dodaje proizvode u korpu i kreira porudžbine. Može da pregleda sopstvene porudžbine i njihove statuse, kao i da uređuje svoj profil. Kreiranje porudžbine zahteva autentifikaciju, dok javni prikaz porekla ostaje dostupan i bez korisničkog naloga.

Vlasnik preduzeća, označen ulogom FARM_OWNER, uspostavlja preduzeće i upravlja njegovim poslovnim podacima. Pored evidencije sirovina, proizvodnih serija i proizvoda, vlasnik dodaje radnike, pregleda njihove naloge, obrađuje porudžbine i pristupa poslovnoj analitici. U postojećem modelu vlasnik može da kreira jedno preduzeće. Prilikom njegovog kreiranja korisnički nalog povezuje se sa preduzećem, a podaci sesije dopunjuju odgovarajućim identifikatorom.

Radnik, sa ulogom WORKER, obavlja operativne poslove u okviru preduzeća kojem pripada. Njegova ovlašćenja obuhvataju rad sa sirovinama, proizvodnim serijama, koracima proizvodnje i proizvodima, kao i dozvoljene promene statusa porudžbina. Radnik ne dodaje druge zaposlene i nema ovlašćenje za otkazivanje porudžbina, koje je dodeljeno vlasniku. Nalog radnika kreira vlasnik preduzeća; ova uloga nije ponuđena kroz javnu registraciju.

Administrator sistema ima ulogu SYSTEM_ADMIN i pristup određenim administrativnim pregledima. Obim njegovih ovlašćenja zavisi od konkretne operacije, pa se ova uloga ne tumači kao neograničeno pravo izmene svih poslovnih podataka. Javna registracija dozvoljava izbor uloge kupca ili vlasnika, čime se sprečava samostalno dodeljivanje radničke ili administratorske uloge.

Pored korisničkih uloga, implementacija koristi tehnička ovlašćenja za međuservisnu komunikaciju. Na primer, TRACEABILITY omogućava ograničeno čitanje podataka o poreklu, dok se posebna ovlašćenja koriste za rezervaciju proizvoda i promenu stanja sirovina. Takva ovlašćenja nisu zasebni poslovni akteri niti nalozi za prijavu kroz korisnički interfejs. Pregled korisničkih uloga prikazan je u tabeli 3.

Uloga	Ključne dozvole	Ograničenje
Posetilac	Javni prikaz porekla i PDF	Bez pristupa internoj evidenciji
CUSTOMER	Ponuda, sopstvene porudžbine i profil	Bez uvida u interne podatke preduzeća
FARM_OWNER	Preduzeće, radnici, proizvodnja, prodaja i analitika	Poslovne operacije u okviru sopstvenog preduzeća
WORKER	Sirovine, proizvodnja, proizvodi i obrada porudžbina	Bez dodavanja radnika i otkazivanja porudžbina
SYSTEM_ADMIN	Dostupni administrativni pregledi	Ovlašćenja zavise od operacije; nema javne registracije

Tabela 3: Uloge i dozvole

3.2 Osnovni pojmovi i odnosi u domenu

Preduzeće predstavlja organizacionu celinu kojoj pripadaju radnici, sirovine, proizvodne serije i proizvodi. Korisnička uloga određuje vrstu dozvoljenih operacija, dok pripadnost preduzeću ograničava skup poslovnih objekata nad kojima se te operacije mogu izvršavati. Kupac se sa preduzećem povezuje kroz porudžbinu, bez sticanja pristupa njenoj internoj evidenciji.

U ovom radu termin preduzeće koristi se u organizacionom smislu i obuhvata porodična gazdinstva, male proizvodne radionice i druge lokalne poslovne celine, uključujući proizvodnju sapuna. Termin ne određuje pravni oblik poslovanja. U korisničkom interfejsu ova celina označena je nazivom Business, a vlasnička uloga nazivom Business owner. Stranica profila razdvaja lične podatke (Personal profile) i poslovne podatke (Business profile). Komanda Add your business dodaje poslovni profil postojećem korisničkom nalogu i ne predstavlja pravnu registraciju firme. Tehničke oznake Farm, FarmOwner, FARM_OWNER, farm_id i farms zadržane su u kodu, API ugovorima i bazi podataka radi kompatibilnosti.

Sirovina predstavlja evidentiranu ulaznu zalihu namenjenu proizvodnji. Osnovni podaci obuhvataju naziv, tip, količinu i jedinicu mere, uz mogućnost navođenja porekla, dobavljača, datuma berbe ili proizvodnje, prijema i isteka roka trajanja. Može se definisati i prag niske zalihe. Katalog tipova sirovina

sadrži zajedničke vrednosti i vrednosti koje korisnici dodaju za sopstveno preduzeće. Ponovljeni unos istog naziva, bez obzira na razliku između velikih i malih slova, vraća postojeći tip.

Proizvodna serija objedinjuje evidenciju jednog proizvodnog toka. Sadrži naziv, datume, napomene, status, korišćene sirovine, proizvodne korake i izlazne proizvode. U aktuelnom modelu seriji se ne dodeljuje poseban tip proizvodnje. Priroda postupka opisuje se nazivom serije, njenim koracima i parametrima tih koraka, što omogućava primenu istog modela na različite proizvodne postupke.

Veza sirovine i serije sadrži utrošenu količinu i istorijski snimak podataka o sirovini. Snimak obuhvata naziv, tip, jedinicu mere, poreklo, dobavljača i evidentirane datume. Time se podaci relevantni za konkretnu proizvodnju čuvaju nezavisno od naknadnog uređivanja izvornog zapisa o sirovini. Ista sirovina ne može se više puta dodati kao zasebna veza unutar iste serije.

Proizvodni korak opisuje pojedinačnu fazu, poput soljenja, dimljenja ili sušenja. Svaki korak ima naziv, redni broj, status, opcioni opis i listu parametara. Parametar se predstavlja parom naziva i tekstualne vrednosti, na primer „Temperatura — 18 °C”, „Trajanje — 48 h” ili „Vlažnost — 75 %”. Nazivi parametara moraju biti jedinstveni unutar koraka, a lista može biti prazna. Ovaj model omogućava evidentiranje različitih uslova procesa, ali ne podrazumeva automatsko tumačenje jedinica mere, proračun parametara ili merenje pomoću senzora.

Jedna proizvodna serija može imati više izlaznih proizvoda. Za svaki izlaz evidentiraju se identitet, naziv, tip proizvoda, količina, jedinica mere i cena, uz mogućnost navođenja dodatnih podataka, poput roka trajanja. Tokom proizvodnje izlazi predstavljaju planirane proizvode. Pri završavanju serije potvrđuju se ostvarene količine, a za izlaze koji su prethodno sačuvani zadržava se i ranije evidentirani plan. Na taj način razlikuju se planirani rezultat proizvodnje i potvrđeni izlaz.

Proizvod predstavlja pojedinačni izlaz serije koji se prati kroz proizvodnju, skladište i prodajnu ponudu. Porudžbina povezuje kupca sa jednim preduzećem i sadrži stavke sa količinama i cenama. Jedan zahtev za kupovinu može obuhvatiti više preduzeća, ali se iz njega formiraju odvojene porudžbine za svakog proizvođača. Naziv proizvoda, jedinica mere i cena čuvaju se u stavci porudžbine kao vrednosti zabeležene pri poručivanju.

3.3 Funkcionalni zahtevi

Funkcionalni zahtevi grupisani su prema poslovnim celinama: upravljanje korisnicima i preduzećima, evidencija sirovina, praćenje proizvodnje, upravljanje proizvodima, poručivanje i prikaz porekla. Zahtev obuhvata očekivano ponašanje sistema i uslove pod kojima se operacija prihvata. Zbog distribuirane organizacije aplikacije, katalog uključuje i funkcije pouzdanog prenosa promena i oporavka podataka za čitanje. Pregled zahteva dat je u tabeli 4.

ID	Zahtev	Očekivano ponašanje i uslovi
F-01	Registracija	Kreiranje naloga kupca ili vlasnika; zaštita lozinke i izdavanje tokena.
F-02	Prijava i sesija	Provera pristupnih podataka i identifikacija korisnika i njegove uloge.
F-03	Korisnički profil	Pregled i izmena sopstvenih podataka uz ograničenja dodeljenih ovlašćenja.
F-04	Preduzeće	Kreiranje jednog preduzeća za vlasnika i upravljanje njegovim podacima.
F-05	Radnici	Vlasnik kreira i pregleda naloge radnika svog preduzeća.
F-06	Evidencija sirovina	Unos, pregled, pretraga, filtriranje, izmena i logičko brisanje sirovina.
F-07	Praćenje zaliha sirovina	Korekcija količina i pregled zaliha prema zadatom pragu.
F-08	Proizvodne serije	Kreiranje i pregled serija sa datumima, napomenama, sirovinama i izlazima.
F-09	Utrošak sirovina	Provera jedinice i zalihe, umanjenje količine i čuvanje istorijskih podataka.
F-10	Proizvodni koraci	Dodavanje, uređivanje i brisanje koraka sa jedinstvenim rednim brojem, dok serija nije zatvorena.
F-11	Završetak proizvodnje	Svi koraci su završeni; postoji najmanje jedan korak i najmanje jedan validan izlaz.
F-12	Skladište proizvoda	Potvrđeni izlazi završene serije prelaze u skladište, bez automatskog aktiviranja prodaje.
F-13	Prodajna ponuda	Aktiviranje zahteva pozitivnu cenu, završenu seriju i važeći rok, ako je naveden.
F-14	Slike proizvoda	Dodavanje slike proizvoda, obrada i prikaz u aplikaciji.
F-15	QR kod	Generisanje i preuzimanje koda za pristup javnoj stranici porekla.
F-16	Prikaz porekla	Povezivanje proizvoda, preduzeća, serije, koraka i istorijskih podataka sirovina.
F-17	Dokument o poreklu	Generisanje PDF dokumenta na osnovu podataka dostupnih javnom prikazu.
F-18	Poručivanje	Rezervacija zaliha i formiranje odvojenih porudžbina za preduzeća iz iste korpe.
F-19	Obrada porudžbine	Kontrolisani prelazi statusa i oslobađanje rezervisane zalihe pri otkazivanju.
F-20	Analitika	Pregled broja i statusa porudžbina, prihoda i najprodavanijih proizvoda.
F-21	Upravljački pregled	Pregled podataka prilagođen ulozi korisnika.
F-22	Izbor proizvođača	Autentifikovani pregled proizvođača radi filtriranja ponude.
F-23	Prenos poslovnih promena	Pouzdan objavljivanje događaja i zaštita od duplirane obrade.
F-24	Obnova projekcija	Ponovno slanje sačuvanih događaja pomoću operativne skripte.
F-25	Katalog tipova sirovina	Zajednički tipovi i dopuna za preduzeće, bez dupliranja istog naziva.
F-26	Parametri koraka	Lista parova naziva i tekstualnih vrednosti; nazivi su jedinstveni unutar koraka.

ID	Zahtev	Očekivano ponašanje i uslovi
F-27	Više proizvodnih izlaza	Planiranje i potvrđivanje više proizvoda i njihovih količina u jednoj seriji.
F-28	Statusi koraka	Planiranje, pokretanje i završetak koraka; pokretanje koraka aktivira seriju.

Tabela 4: Katalog funkcionalnih zahteva

Zahtevi F-25 do F-28 posebno izdvajaju proširivost kataloga sirovina, prilagodljive parametre koraka, podršku za više izlaznih proizvoda i upravljanje statusima koraka. Proširivost nije jednaka za sve poslovne objekte: tip sirovine može se dopuniti u okviru preduzeća, dok se tipovi izlaznih proizvoda pri završavanju serije proveravaju prema unapred definisanom skupu. Podržani tipovi izlaza su meat, dairy, vegetable, fruit, cheese, sausage, honey i other, a jedinice mere kg, g, l, ml i pcs.

3.4 Poslovna pravila i životni ciklusi

Proizvodna serija nastaje u statusu PLANNED. Pokretanje bilo kog njenog koraka, prelaskom koraka u IN_PROGRESS, automatski postavlja seriju u IN_PROGRESS i beleži datum početka ukoliko ranije nije naveden. Korak zatim može preći u COMPLETED. Preskakanje početnog stanja ili vraćanje koraka u prethodni status nije podržano. Redni brojevi koraka moraju biti pozitivni i jedinstveni unutar serije; oni određuju redosled prikaza, ali implementacija ne zahteva da se svi koraci pokreću strogo prema tim brojevima.

Završetak svih koraka ne završava automatski proizvodnu seriju. Korisnik zasebno potvrđuje završetak proizvodnje i konačne izlaze. Prelazak serije u COMPLETED zahteva najmanje jedan korak, završene sve postojeće korake i najmanje jedan izlazni proizvod. Za svaki izlaz proveravaju se naziv, podržani tip i jedinica mere, pozitivna količina i nenegativna cena. Serija može biti otkazana iz planiranog ili aktivnog stanja, prelaskom u CANCELLED. Završene i otkazane serije ne prihvataju izmene kroz operaciju ažuriranja serije, a njihovi koraci ne mogu se dodavati, menjati ili brisati.

Životni ciklus proizvoda razlikuje stanja PRODUCTION, STORAGE i ON_SALE. Planirani izlazi prikazuju se kao proizvodi u proizvodnji i uređuju kroz odgovarajuću seriju. Nakon potvrđenog završetka serije prelaze u skladište, pri čemu ne postaju automatski dostupni za kupovinu. Stavljanje u prodaju zahteva pozitivnu cenu, završenu pripadajuću seriju i važeći rok trajanja, kada je naveden. Proizvod se ne kreira nezavisno od proizvodnje kroz uobičajenu operaciju dodavanja proizvoda. Nakon prenosa u skladište, količina potvrđenog proizvodnog izlaza, jedinica mere i pripadnost seriji ne mogu se proizvoljno menjati kroz uređivanje proizvoda.

Pri evidentiranju utroška sirovine zahteva se pozitivna količina, najmanje 0,001, sa najviše tri decimale, usklađena jedinica mere i dovoljno raspoložive zalihe. Nije podržana automatska konverzija između jedinica, pa količina navedena u gramima ne može neposredno umanjiti zalihu evidentiranu u

kilogramima. Umanjenje stanja štiti se od konkurentnih zahteva i ponavljanja iste operacije. Dozvoljeno uklanjanje veze sirovine iz serije upisuje trajni zadatak za povraćaj ranije utrošene količine. Takvi zadaci nastaju i pri otkazivanju ili logičkom brisanju serije, a pozadinski proces ih izvršava preko servisa sirovina. Povraćaj je zato asinhron i treba ga razlikovati od trenutne promene lokalne evidencije.

Porudžbina nastaje u stanju PENDING, nakon uspešne rezervacije potrebnih količina. Podržani redovni tok je PENDING, CONFIRMED, SHIPPED i DELIVERED. Otkazivanje je dozvoljeno iz stanja PENDING ili CONFIRMED i rezervisano je za vlasnika preduzeća. Otkazivanje upisuje trajni zadatak za oslobađanje rezervisane zalihe, čije izvršavanje može biti ponovljeno ako servis proizvoda privremeno nije dostupan. Logičko brisanje dozvoljeno je samo za porudžbine koje čekaju obradu ili su otkazane, uz odgovarajuće oslobađanje zalihe.

Pri poručivanju se proveravaju dostupnost proizvoda, tražena količina i cena korišćena za rezervaciju. Stavke koje se odnose na isti proizvod objedinjuju se, a količine moraju biti pozitivne i imati najviše tri decimale. Ponovljeni zahtev sa istim ključem idempotentnosti i istim sadržajem nastavlja postojeću obradu ili vraća njen sačuvani rezultat. Upotreba istog ključa za drugog korisnika ili drugačiji zahtev odbija se kao konflikt. Time se smanjuje mogućnost da ponavljanje zahteva nakon prekida komunikacije proizvede duplu porudžbinu ili višestruko umanjenje zalihe.

Promene proizvoda nastale iz proizvodnih događaja i objedinjeni prikaz porekla ažuriraju se asinhrono. Zbog toga uspešan završetak operacije u jednom servisu ne podrazumeva da su svi povezani prikazi istog trenutka osveženi. Sistem mora razlikovati privremeno neusaglašene podatke od nepostojećeg ili nedostupnog proizvoda. Trenutno stanje prodajne dostupnosti proverava se u servisu proizvoda, dok se povezani podaci o poreklu preuzimaju iz modela za čitanje.

3.5 Glavni slučajevi korišćenja

3.5.1 Uspostavljanje preduzeća i dodavanje radnika

Preduslov je prijavljen korisnik sa ulogom vlasnika koji još nema preduzeće. Vlasnik unosi naziv i adresu, uz opcione kontaktne i opisne podatke. Sistem proverava postojeću povezanost vlasnika sa preduzećem, kreira zapis i izdaje osveženi token sa identifikatorom preduzeća. Nakon toga vlasnik može da doda radnika unosom podataka potrebnih za njegov nalog. Sistem proverava pripadnost preduzeća vlasniku i jedinstvenost adrese elektronske pošte. Ishod je preduzeće sa povezanim korisnicima i ovlašćenjima potrebnim za vođenje evidencije.

3.5.2 Evidentiranje sirovina i planiranje proizvodnje

Vlasnik ili radnik evidentira novu sirovinu unosom osnovnih podataka o zalihi i, po potrebi, podataka o poreklu i datumima. Tip bira iz dostupnog kataloga ili dodaje novu vrednost za svoje preduzeće. Zatim kreira proizvodnu seriju i bira sirovine koje će se koristiti, sa odgovarajućim količinama i jedinicama mere. Sistem proverava pripadnost sirovina preduzeću i raspoloživost zalihe, evidentira utrošak i čuva istorijske podatke u seriji. Utrošak se evidentira već pri povezivanju sirovine sa serijom, a ne tek pri njenom završavanju. Nedovoljna količina ili neusaglašena jedinica mere dovode do odbijanja operacije.

Planiranje se nastavlja dodavanjem proizvodnih koraka i izlaznih proizvoda. Korisnik određuje nazive i redne brojeve koraka, unosi opis postupka i parametre koje želi da prati. Za izlaze definiše očekivane proizvode i njihove količine. Ishod je serija u planiranom stanju sa povezanim ulazima, opisanim fazama rada i evidentiranim očekivanim rezultatima.

3.5.3 Izvršavanje koraka i potvrđivanje proizvodnog izlaza

Korisnik pokreće proizvodni korak, čime se aktivira i serija. Tokom rada evidentira relevantne parametre i završava pojedinačne korake promenom njihovog statusa. Kada su svi koraci završeni, potvrđuje konačne proizvode i ostvarene količine. Sistem odbija završetak ako nema koraka, ako neki korak nije završen ili ako izlazni podaci ne zadovoljavaju propisana pravila. Nakon prihvatanja, serija dobija status COMPLETED, a događaj o promeni omogućava prenos izlaznih proizvoda u skladište.

Na primer, jedna serija može obuhvatiti više gotovih proizvoda nastalih iz istog proizvodnog postupka. Svaki izlaz zadržava sopstveni identifikator i količinu, dok se svi povezuju sa zajedničkom serijom i njenim sirovinama i koracima. Zaštita od ponovne obrade istog ili starijeg događaja sprečava da višestruka isporuka poruke proizvoljno uveća evidentiranu zalihu. Zbog asinhronne obrade, prikaz skladišta može postati ažuran naknadno.

3.5.4 Priprema proizvoda za prodaju

Preduslov je proizvod prenet iz završene serije u skladište. Vlasnik ili radnik uređuje prodajne podatke, postavlja pozitivnu cenu i po potrebi dodaje opis i sliku. Zahtev za prelazak u prodajnu ponudu pokreće proveru završetka proizvodnje, pripadnosti preduzeću i roka trajanja. Ako su uslovi ispunjeni, proizvod dobija stanje ON_SALE. U suprotnom, sistem odbija aktiviranje i proizvod ostaje van prodajne ponude. Planirani proizvodi ne mogu se aktivirati kao gotova roba pre završetka serije.

3.5.5 Kreiranje i obrada porudžbine

Prijavljeni kupac bira proizvode i količine i potvrđuje sadržaj korpe. Sistem proverava stavke, čuva podatke potrebne za nastavak obrade i zahteva rezervaciju zalihe. Nakon uspešne rezervacije stavke grupiše prema preduzeću i formira odgovarajuće porudžbine, sa zabeleženim cenama. Ako je proizvod postao nedostupan, zaliha nije dovoljna ili se cena promenila, zahtev se ne prihvata kao uspešno završena kupovina. Pri privremenom prekidu komunikacije obrada može biti nastavljena istim identifikatorom zahteva.

Vlasnik ili radnik zatim menja status porudžbine u skladu sa dozvoljenim prelazima. Kupac prati status sopstvenih porudžbina, dok vlasnik može da otkáže porudžbinu pre slanja. Otkazivanje pokreće oslobađanje prethodno rezervisanih količina. Oznake slanja i isporuke predstavljaju ručno evidentirane poslovne statuse; u aktuelnom rešenju one nisu potvrde dobijene od integrisane kurirske službe niti dokaz izvršenog plaćanja.

3.5.6 Pregled porekla proizvoda

Posetilac skenira QR kod i otvara javnu stranicu odgovarajućeg proizvoda. Sistem proverava token i trenutnu dostupnost proizvoda, a zatim objedinjuje podatke o preduzeću, proizvodnoj seriji, njenim koracima i korišćenim sirovinama. Posetilac može da pregleda podatke i preuzme dokument o poreklu u PDF formatu. Ako projekcija još nije usaglašena, prikazuje se odgovarajuće obaveštenje ili raspoloživi nepotpuni podaci, uz mogućnost ponovnog pokušaja. Prikaz se zasniva na evidentiranim informacijama i ne dopunjava nedostajuće podatke pretpostavkama.

3.5.7 Pregled poslovnih pokazatelja

Vlasnik pristupa analitičkom pregledu za svoje preduzeće i izabrani period. Sistem prikazuje broj porudžbina, raspodelu po statusima, mesečne iznose i pregled najprodavanijih proizvoda. Prihod se računa na osnovu porudžbina označenih kao isporučene, pa predstavlja pokazatelj izveden iz evidencije prodaje, a ne potvrdu finansijske naplate. Upravljački pregled dopunjuje operativnu evidenciju i omogućava sagledavanje rezultata bez pojedinačnog otvaranja svake porudžbine.

3.6 Ograničenja funkcionalnog obuhvata

Realizovani model omogućava prilagođavanje proizvodnje kroz nazive serija, uređene korake, tekstualne parametre i više izlaznih proizvoda. Međutim, ne obuhvata automatski proračun prinosa, obaveznu jednakost ulazne i izlazne mase, recepture sa normativima niti automatsku konverziju jedinica mere. Parametri procesa evidentiraju se ručno i ne predstavljaju automatski prikupljena merenja. Ova

ograničenja određuju granicu između evidencije proizvodnje i sistema za automatizovano upravljanje tehnološkim procesom.

Poručivanje je podržano kao evidencija zahteva kupca, rezervacije zalihe i promena poslovnog statusa. Elektronsko plaćanje, fiskalizacija i integracija dostave nisu deo ovog toka. Takođe, podaci o poreklu predstavljaju sledljivost zapisa koje su uneli ovlašćeni korisnici, bez nezavisne sertifikacije njihove tačnosti. Ovakvo razgraničenje omogućava da se ostvarene funkcionalnosti procenjuju prema stvarnom obuhvatu implementacije.

4 Teorijske osnove i izbor tehnologija

Tehnološka osnova platforme LocalBite oblikovana je prema zahtevima za razdvajanjem poslovnih odgovornosti, kontrolisanim pristupom podacima, praćenjem proizvodnje i pouzdanom razmenom promena između servisa. Serverski deo razvijen je u programskom jeziku Rust, korisnički interfejs u Angular okruženju, a trajno čuvanje podataka i asinhrona komunikacija ostvareni su pomoću PostgreSQL baza i RabbitMQ brokera. U nastavku su predstavljene uloge ovih tehnologija i način njihove primene u projektu.

Navedene verzije odnose se na deklaracije u projektnim manifestima i oznake kontejnerskih slika. Konačne verzije zavisnosti određuju datoteke Cargo.lock i package-lock.json. Oznaka Rust 2021 predstavlja izdanje jezika, a ne verziju kompajlera. Razlikovanje ovih podataka važno je za ponovljivost izgradnje i pravilno tumačenje razvojnog okruženja.

4.1 Mikroservisna arhitektura

Mikroservisna arhitektura zasniva se na podeli aplikacije na zasebne servise sa jasno određenim odgovornostima i komunikacijom preko definisanih interfejsa [5]. LocalBite obuhvata pet poslovnih servisa za korisnike i preduzeća, sirovine, proizvodnju, proizvode i porudžbine, kao i servis namenjen projekcijama podataka za čitanje. Svaki servis ima sopstveni izvršni proces i bazu podataka.

Ovakva organizacija omogućava da se pravila proizvodnje razvijaju odvojeno od pravila poručivanja ili prikaza porekla. Istovremeno, operacija koja obuhvata više servisa ne može se osloniti na jednu lokalnu SQL transakciju. Na primer, rezervacija proizvoda i upis porudžbine zahtevaju koordinaciju dve baze. Zbog toga su pouzdano objavljivanje događaja, idempotentnost i oporavak sastavni delovi rešenja. Zajednički repozitorijum i biblioteka common olakšavaju razvoj, ali uvode i određenu povezanost servisa pri izgradnji i promeni zajedničkih ugovora.

4.2 Rust i upravljanje projektom pomoću Cargo alata

Rust je statički tipizovan jezik u kojem se upravljanje memorijom zasniva na vlasništvu nad vrednostima i pravilima pozajmljivanja referenci. Ova pravila omogućavaju otkrivanje određenih grešaka pri prevođenju programa, bez oslanjanja na sakupljač otpada [10]. U projektu tipovi Option i Result eksplicitno predstavljaju odsustvo vrednosti i ishod operacije, što je značajno za obradu opcionih datuma, rezultata upita i poslovnih grešaka.

Serverski projekat organizovan je kao Cargo workspace sa šest servisnih paketa i zajedničkom bibliotekom. Zavisnosti koje koristi više servisa deklarisan su u korenskom manifestu, dok svaki servis zadržava svoj ulazni program, modele, migracije i konfiguraciju. Biblioteka common objedinjuje, između

ostalog, obradu grešaka, proveru ovlašćenja i infrastrukturu integracionih događaja. Time se smanjuje ponavljanje koda koji mora imati isto značenje u različitim servisima.

Deljene instance poslovnih servisa i repozitorijuma prosleđuju se pomoću Arc tipa. Takvo deljenje objekata unutar procesa ne zamenjuje kontrolu konkurentnosti u bazi: istovremene izmene zaliha i dalje zahtevaju transakcije, uslovne upite ili zaključavanje. Izgradnja u kontejneru koristi postojeću datoteku Cargo.lock i optimizovani režim prevođenja, čime se olakšava ponavljanje izgradnje sa istim skupom zavisnosti.

4.3 Axum i Tokio

Axum je okvir za razvoj HTTP aplikacija koji povezuje putanje i metode zahteva sa funkcijama za njihovu obradu. Mehanizam izdvajanja podataka iz zahteva omogućava tipizovano čitanje putanje, parametara upita i JSON sadržaja [13]. U projektu je deklarirana verzija Axum 0.8.8. Identifikatori se preuzimaju kroz Path<Uuid>, filteri i paginacija kroz Query, a podaci obrazaca kroz Json strukture. Projektni AuthClaims mehanizam izdvaja i proverava podatke iz pristupnog tokena.

Funkcija za obradu zahteva povezuje HTTP interfejs sa poslovnim slojem. Na primer, zahtev za dodavanje sirovine dobija identitet preduzeća iz proverene sesije, umesto da prihvati proizvoljan identifikator poslat iz obrasca. Zajednički tip AppError prevodi poslovne i infrastrukturne greške u odgovarajuće HTTP odgovore. Ovakvo razdvajanje olakšava ujednačeno prijavljivanje neispravnog zahteva, zabrane pristupa, nepostojećeg resursa ili konflikta.

Tokio obezbeđuje izvršno okruženje za asinhronu operacije u Rust-u [14]. Koristi se za prihvatanje mrežnih zahteva, pristup bazi, međuservisne pozive i pozadinske zadatke. Dok operacija čeka mrežni odgovor ili rezultat upita, izvršno okruženje može obrađivati druge spremne zadatke. U LocalBite-u pozadinski zadaci objavljuju događaje, primaju poruke i ponavljaju nedovršene operacije oporavka. Asinhrono izvršavanje samo po sebi ne obezbeđuje poslovnu ispravnost, pa su provere stanja i transakcione granice definisane zasebno.

4.4 HTTP interfejsi i međuservisna komunikacija

REST predstavlja arhitektonski stil za distribuirane hipermedijske sisteme [6]. U projektu se resursno organizovani HTTP interfejsi koriste za operacije nad korisnicima, preduzećima, sirovinama, serijama, proizvodima i porudžbinama. Metoda i putanja određuju traženu operaciju, dok ulazne i izlazne strukture definišu ugovor između klijenta i servisa. HTTP komunikacija naročito je značajna kada je pre nastavka poslovnog toka potreban neposredan rezultat provere.

Biblioteka Reqwest koristi se za pozive između servisa. Servis proizvodnje preko nje pribavlja podatke o sirovinama i evidentira njihovu potrošnju, servis porudžbina zahteva rezervaciju proizvoda, a servis proizvoda pribavlja projekciju porekla. Adrese odredišta podešavaju se kroz promenljive okruženja. Interni pozivi koriste odgovarajuća tehnička ovlašćenja, dok vremenska ograničenja sprečavaju neograničeno čekanje na nedostupan servis.

Prekid HTTP komunikacije ne određuje pouzdano da li je udaljena operacija izvršena. Zbog toga zahtev za kreiranje porudžbine koristi ključ idempotentnosti, a rezervacija se povezuje sa trajno sačuvanim identifikatorom obrade. Ponovljeni poziv nastavlja istu operaciju ili vraća njen rezultat. Ovaj mehanizam je deo poslovnog protokola i ne proizlazi automatski iz upotrebe HTTP-a.

4.5 PostgreSQL i SQLx

PostgreSQL je relacioni sistem za upravljanje bazama podataka sa podrškom za transakcije, integritet podataka i proširive tipove [8]. Konfiguracija projekta koristi PostgreSQL 15 i zasebnu bazu za svaki servis. Takav izbor omogućava da se ograničenja i migracije šeme vezuju za servis koji poseduje podatke. Veze između različitih servisnih baza proveravaju se kroz aplikacionu komunikaciju, jer ne postoje zajednički međubazni strani ključevi.

U modelu podataka koriste se UUID identifikatori, decimalne količine i novčani iznosi, datumi i JSONB strukture. Ograničenja CHECK i UNIQUE sprečavaju pojedine nedozvoljene vrednosti i duplikate. Transakcije, zaključavanje redova i uslovne izmene štite konkurentne operacije nad zalihama. Okidači baze dodatno obezbeđuju upis integracionih događaja i zabranu izmena zatvorenih proizvodnih serija. Time se deo važnih pravila sprovodi i na nivou trajnog čuvanja podataka.

SQLx, deklarisan u verziji 0.8.6, omogućava asinhroni pristup PostgreSQL-u uz eksplicitne SQL upite [15]. U projektu se koriste skupovi konekcija, parametrizovani upiti i migracije koje se izvršavaju pri pokretanju servisa. Pojedini upiti koriste makroe za proveru pri prevođenju i sačuvane metapodatke iz direktorijuma .sqlx, dok se dinamički upiti proveravaju pri izvršavanju. Stoga se proveru pri prevođenju ne može pripisati svakom upitu u aplikaciji.

Kombinacija PostgreSQL-a i SQLx-a omogućava neposrednu kontrolu nad transakcijama i složenijim upitima potrebnim za rezervacije i projekcije. Cena takvog pristupa je obaveza ručnog održavanja SQL izraza, mapiranja rezultata i usklađenosti sa migracijama. SQLx se u ovom radu koristi kao biblioteka za pristup bazi, a ne kao objektno-relacioni okvir koji automatski upravlja celim životnim ciklusom poslovnih objekata.

4.6 RabbitMQ i Lapin

RabbitMQ je posrednik za razmenu poruka kojim su u projektu povezani proizvođači i potrošači integracionih događaja. Biblioteka Lapin, sa eksplicitno navedenom verzijom 2.5.5, omogućava Rust servisima komunikaciju preko AMQP protokola. Topologija sadrži razmenjivač poruka tipa topic, poseban red za projekcije i poseban red za obradu izlaza proizvodnje. Na taj način jedna promena serije može biti nezavisno obrađena radi ažuriranja porekla i radi formiranja proizvoda.

Pouzdanost se oslanja na trajne redove, poruke označene za trajno čuvanje, potvrde objavljivanja i potvrde uspešne obrade. Potvrda brokera odnosi se na objavljenu poruku, dok potvrda potrošača označava završetak obrade na njegovoj strani; to su različite odgovornosti [9]. U projektu se događaj označava kao objavljen tek nakon potvrde, a potrošač projekcije potvrđuje prijem tek nakon potvrđivanja lokalne transakcije.

Ponovna isporuka poruke ostaje moguća, na primer ako je obrada završena neposredno pre prekida veze. Zato RabbitMQ nije dovoljan bez aplikacionog prepoznavanja duplikata. Ovaj odnos povezuje izbor brokera sa obrascima Transactional Outbox i Idempotent Consumer, opisanim u odeljku 5.6. Asinhrona komunikacija smanjuje vremensku zavisnost servisa, ali zahteva obradu privremeno neusaglašenih prikaza i praćenje neuspešnih poruka.

4.7 Serijalizacija i pomoćni tipovi podataka

Serde obezbeđuje serijalizaciju i deserijalizaciju struktura, dok `serde_json` podržava JSON format [16]. U LocalBite-u ovi alati povezuju HTTP zahteve, odgovore i integracione događaje sa Rust tipovima. Namenske strukture za prenos podataka odvojene su od modela baze. Na primer, odgovor za detalje serije objedinjuje podatke o seriji, koracima i sirovinama koji se trajno čuvaju odvojeno.

Biblioteke `uuid`, `chrono` i `bigdecimal` koriste se za identifikatore, vremenske vrednosti i decimalne brojeve. UUID omogućava stvaranje identiteta objekata i operacija unutar različitih servisa. Chrono razlikuje datume i vremenske zapise, dok BigDecimal podržava rad sa decimalnim vrednostima iz baze. Pošto pojedini HTTP ugovori koriste brojeve tipa `f64`, ograničenja preciznosti i pretvaranja vrednosti moraju se proveravati na granici sistema; sam izbor decimalnog tipa u bazi ne uklanja sve rizike zaokruživanja.

4.8 Autentifikacija i zaštita pristupa

Identitet korisnika prenosi se pomoću JSON Web Token formata [7]. Token u projektu sadrži identifikator korisnika, adresu elektronske pošte, ulogu, pripadnost preduzeću i podatke o vremenu

važenja. Klijent ga dostavlja u zaglavlju Authorization, a server proverava token pre izvršavanja zaštićene operacije. Poslovna autorizacija zatim proverava ulogu i pripadnost traženog objekta preduzeću.

Lozinke se obrađuju algoritmom Argon2, uz nasumičnu so. Argon2 je namenjen heširanju lozinki sa troškom rada i memorije koji otežava masovno isprobavanje kandidata [17]. U bazi se čuva rezultat heširanja, a pri prijavi proverava se uneta lozinka. To je različito od reverzibilnog šifrovanja. Biblioteka jsonwebtoken obrađuje tokene, dok argon2 obezbeđuje odvojenu odgovornost zaštite lozinki.

Servisi koriste zajedničku simetričnu tajnu za tokene, što pojednostavljuje konfiguraciju, ali proširuje granicu poverenja između njih. Tehnički tokeni služe ograničenim internim operacijama, kao što je čitanje porekla. Provere u Angular interfejsu upravljaju navigacijom i vidljivošću akcija, ali ne zamenjuju obavezne serverske provere.

4.9 Angular i TypeScript

Angular omogućava organizaciju korisničkog interfejsa kroz komponente, rutiranje, obrasce i ubrizgavanje zavisnosti [11]. Projekat koristi Angular 21, dok je za TypeScript deklarisan opseg verzije 5.9. TypeScript dopunjuje JavaScript statičkim opisima tipova [18], što olakšava definisanje modela korisnika, proizvodnih serija, proizvoda i odgovora servisa.

Interfejs je organizovan kao aplikacija sa jednom stranicom i samostalnim komponentama. Funkcionalne celine učitavaju se kroz rute, a odloženo učitavanje omogućava da se određeni delovi aplikacije pribave tek pri navigaciji. Reaktivni obrasci povezuju unos podataka sa validacijom, dok zajednički prikazi grešaka ujednačavaju povratne informacije korisniku. Angular Material je prisutan među zavisnostima, ali značajan deo izgleda ostvaren je sopstvenim HTML i CSS rešenjima.

Stanje komponentata prati se pomoću signala i izvedenih vrednosti. Promena izabranog filtera, statusa učitavanja ili podataka serije tako se povezuje sa osvežavanjem prikaza. API servisi izdvajaju komunikaciju sa serverskim delom, a HTTP interceptor dodaje pristupni token. Ovakva organizacija smanjuje ponavljanje mrežnog i autentifikacionog koda u pojedinačnim ekranima.

4.10 RxJS i reaktivna obrada

RxJS omogućava predstavljanje tokova vrednosti i događaja kroz Observable objekte i njihovu obradu pomoću operatora [19]. U projektu Angular HttpClient vraća ovakve tokove, pa se odgovori mogu transformisati, povezati sa prikazom greške i obraditi nakon prijema. Reaktivna pretplata koristi se i za periodično osvežavanje podataka.

Upravljački pregled se osvežava u intervalima od pet sekundi, uz prekid pretplate kada komponenta prestane da se koristi. Javni prikaz porekla može ograničeno ponavljati zahtev kada dobije odgovor da se

projekcija još usaglašava. Ovi primeri pokazuju kako je asinhrona priroda servera predstavljena u interfejsu. Ponavljanje operacije koja menja podatke, međutim, zahteva zasebnu zaštitu idempotentnosti i ne može se bezuslovno primeniti na svaki zahtev.

4.11 Obrada slika, QR kodova i PDF dokumenata

Biblioteka `image` koristi se za obradu fotografija proizvoda, dok `qrcode` generiše kod koji sadrži adresu javnog prikaza porekla. Obrada fotografije obuhvata prilagođavanje dimenzija i čuvanje u JPEG formatu. Slike i QR kodovi smeštaju se u direktorijum servisa proizvoda povezan sa trajnim volumenom, a baza čuva njihove putanje. QR kod je sredstvo pristupa podacima i ne predstavlja digitalni potpis niti nezavisnu potvrdu njihovog sadržaja.

Dokument o poreklu generiše se u Python-u pomoću `ReportLab` biblioteke. Njen `Platypus` sloj omogućava sastavljanje dokumenta od paragrafa, tabela i drugih elemenata [20]. Servis proizvoda priprema javni skup podataka, prosleđuje JSON pomoćnom procesu preko standardnog ulaza i preuzima PDF preko standardnog izlaza. Python proces je pomoćna komponenta istog servisa, a ne poseban poslovni mikroservis.

Generisanje dokumenta organizovano je primenom obrasca `Builder`, tako da su redosled sastavljanja sadržaja i konkretan način PDF prikaza razdvojeni. Tekst koji korisnik unosi obrađuje se pre prosleđivanja elementima dokumenta, a trajanje procesa je ograničeno. Ovakva kombinacija omogućava da osnovna poslovna aplikacija ostane u Rust-u, dok se za raspored sadržaja dokumenta koristi namenska biblioteka.

4.12 Docker Compose i Nginx

Docker Compose opisuje više povezanih kontejnera, njihove mreže, volumene i konfiguraciju u zajedničkoj YAML datoteci [21]. U projektu se koristi za pokretanje servisa, baza, brokera, korisničkog interfejsa i ulaznog posrednika. Višefazna izgradnja odvaja alate za prevođenje od izvršne slike. Frontend se gradi pomoću Node.js okruženja i npm alata, nakon čega se dobijene statičke datoteke poslužuju preko Nginx-a.

Nginx ima ulogu obrnutog posrednika koji prosleđuje HTTP zahteve odgovarajućem odredištu [22]. Korisnički interfejs koristi zajednički prefiks `/api`, dok konfiguracija određuje nadležni servis. Time se unutrašnje adrese servisa izdvajaju iz klijentskih komponentata. Javni QR pristup koristi ograničen skup ruta; pomoćna skripta može obezbediti privremeni javni pristup radi demonstracije.

Kontejnerizacija pojednostavljuje uspostavljanje razvojnog okruženja, ali postojeća konfiguracija sama po sebi ne dokazuje visoku raspoloživost. Više replika, upravljanje tajnama, deljeno skladište

datoteka i oporavak infrastrukture zahtevaju dodatno planiranje. U okviru rada Compose prvenstveno predstavlja ponovljiv način pokretanja i integracione provere sistema.

4.13 Alati za proveru i dijagnostiku

Provera sistema obuhvata Rust testove, frontend testove zasnovane na Vitest i jsdom zavisnostima, kao i integracione skripte u Node.js, Python i PowerShell okruženjima. Skripte proveravaju poslovne tokove poput rezervacije zaliha, nastanka proizvoda, obnove projekcija i generisanja dokumenta. Jedinični test i provera celog distribuiranog toka imaju različite ciljeve: prvi izdvaja lokalno pravilo, dok drugi proverava saradnju procesa i baza.

Biblioteke tracing i tracing-subscriber koriste se za strukturisano beleženje događaja i grešaka, uz podešavanje nivoa zapisa kroz okruženje. Takvi zapisi pomažu dijagnostici, ali ne predstavljaju potpunu infrastrukturu za distribuirano praćenje. Pregled osnovnih tehnologija i njihovih odgovornosti dat je u tabeli 5, dok je detaljnija analiza provere sistema izložena u poglavlju 11.

Tehnologija	Primena	Značaj za rešenje
Rust i Cargo	Serverski servisi i zajednička biblioteka	Tipizovani modeli i objedinjeno upravljanje zavisnostima
Axum i Tokio	HTTP obrada i asinhrono izvršavanje	Tipizovani zahtevi i konkurentne ulazno-izlazne operacije
PostgreSQL 15	Zasebne servisne baze	Transakcije, integritet i kontrola konkurentnih izmena
SQLx	SQL pristup i migracije	Eksplisitni upiti i kontrola granica transakcije
RabbitMQ i Lapin	Razmena integracionih događaja	Nezavisna obrada promena uz potvrde i ponavljanje
Reqwest i Serde	HTTP pozivi i prenos podataka	Komunikacija kroz strukturisane JSON ugovore
JWT i Argon2	Identitet i zaštita lozinki	Provera pristupa i čuvanje heširanih lozinki
Angular i TypeScript	Komponente, rute i obrasci	Organizacija i tipizacija korisničkog interfejsa
RxJS	Tokovi odgovora i događaja	Transformacija rezultata i kontrolisano osvežavanje
image i qrcode	Fotografije i QR kodovi	Obrada medija i pristup poreklu proizvoda
Python i ReportLab	Dokument o poreklu	Programsko sastavljanje PDF prikaza
Docker Compose i Nginx	Pokretanje i prosleđivanje zahteva	Ponovljivo okruženje i zajednička ulazna tačka
Vitest i pomoćne skripte	Provera komponenti i poslovnih tokova	Izdvojene i integracione provere

Tabela 5: Tehnologije i njihova primena

5 Arhitektura sistema i primenjeni softverski obrasci

5.1 Kontekst

Korisnički interfejs je Angular SPA. Nginx predstavlja ulaznu tačku i prosleđuje relativne /api putanje odgovarajućim backend servisima. Krajni korisnik ne pristupa direktno PostgreSQL bazama ni RabbitMQ brokeru. Javni QR tok izložen je kroz ograničeni skup ruta, dok zaštićeni deo koristi JWT.

Komponenta	Odgovornost	Veza
Angular UI	ekrani, forme, sesija, dozvole	HTTP prema Nginx-u
Nginx	gateway i javni ulaz	prosleđivanje ruta
Auth	identitet i preduzeća	REST i JWT
Raw Materials	sirovine i potrošnja	REST, outbox
Productions	serije i procesi	REST, outbox
Products	zaliha, ponuda, QR	REST, RabbitMQ
Orders	porudžbine i analitika	REST, kompenzacije
Read Models	projekcije i složena čitanja	RabbitMQ consumer

Tabela 6: Servisne granice

5.2 Servisne granice i vlasništvo

Auth poseduje korisnike, uloge, profil preduzeća i vezu korisnika sa preduzećem. Raw Materials poseduje raspoložive sirovine i ledger potrošnje. Productions poseduje serije, korake, istorijske snimke i izlaz proizvodnje. Products poseduje gotovu zalihu, cenu, aktivnost, medije, QR i projekciju stanja serije. Orders poseduje porudžbine, stavke, status, istorijske cene i poslove oporavka. Read Models poseduje kopije podataka namenjene projekcijama.

5.3 Slojevita organizacija

Unutar većine servisa tok ide od router-a i extractors sloja, preko handler-a i service sloja, do repository sloja i baze. Handler obrađuje HTTP detalje, korisnika i opseg podataka preduzeća. Service sadrži poslovna pravila i koordinaciju. Repository izvršava SQL i upravlja lokalnom transakcijom. Izuzeci postoje tamo gde je istorijski tok implementacije direktniji, što je dokumentovano kao granica stroge Clean ili Hexagonal arhitekture.

```
Router -> Handler -> Service -> Repository -> PostgreSQL
|
+--> HTTP client drugog servisa
```

5.4 Arhitektonske odluke

Zasebna baza po servisu daje jasno vlasništvo i lokalnu evoluciju šeme, ali ukida međuservisni strani ključ. Snapshot poruke pojednostavljaju projekciju i replay, ali povećavaju veličinu payload-a. Outbox kroz database trigger pokriva sve SQL putanje, dok aplikacioni relay uvodi asinhronost između

commit-a i objavljivanja. Lokalni uploads volume olakšava razvoj, ali nije dovoljan za horizontalno skaliranje replika.

5.5 Deployment model

Docker Compose pokreće PostgreSQL instance, RabbitMQ, servise, frontend i Nginx. Razvojni portovi i konfiguracija namenjeni su lokalnom okruženju. Za produkciju bi bilo potrebno odvojiti tajne, ograničiti izložene portove, obezbediti replikaciju brokera, deljeni objektni storage, centralne metrike i kontrolisano upravljanje migracijama.

5.6 Primenjeni softverski obrasci

Softverski obrasci predstavljaju ponovljiva rešenja za karakteristične probleme projektovanja. U LocalBite-u njihova primena obuhvata organizaciju koda, izbor ponašanja, izgradnju dokumenata i saradnju distribuiranih servisa. Klasični objektni obrasci, kao što su Strategy i Builder [23], razlikuju se od arhitektonskih i integracionih obrazaca. Zato se svaki obrazac u nastavku povezuje sa konkretnim problemom, učesnicima u implementaciji i granicama ostvarene primene.

5.6.1 Repository i Service Layer

Repository izdvađa pristup trajno sačuvanim podacima iza operacija koje su razumljive poslovnom sloju [24]. U projektu su primeri BatchRepository, ProductRepository i OrderRepository. Metoda `find_by_id_and_farm` izražava potrebu za pronalaženjem objekta u okviru preduzeća, dok njen SQL izraz ostaje u repozitorijumu. Pozivalac tako ne mora u svakom poslovnom toku ponovo da sastavlja uslove filtriranja i mapiranje rezultata.

Service Layer objedinjuje poslovne operacije i koordinira njihov tok [25]. BatchService, na primer, proverava da li je serija završena, da li svi koraci dozvoljavaju završetak proizvodnje i da li su izlazni proizvodi validni. Tek nakon provera prosleđuje izmenu repozitorijumu. HTTP funkcija ostaje odgovorna za prijem zahteva i ovlašćenja, dok je poslovna odluka izdvojena iz mrežnog interfejsa.

Ova kombinacija smanjuje ponavljanje i olakšava pronalaženje pravila pri održavanju. Ipak, repozitorijumi su pretežno konkretne Rust strukture, a deo integracionih tokova izvršava SQL neposredno. Primena je stoga slojevit organizacija sa poznatim izuzecima, bez tvrdnje da je ostvarena potpuna nezavisnost domena od infrastrukture ili stroga heksagonalna arhitektura.

5.6.2 Ubrizgavanje zavisnosti

Ubrizgavanje zavisnosti razdvaja stvaranje saradničkih objekata od njihove upotrebe [26]. U Rust servisima početna funkcija konstruiše skup konekcija, repozitorijume i poslovne servise, a zatim ih povezuje kroz konstruktore. Na primer, BatchService dobija repozitorijume serija, koraka i sirovina, kao i klijent za operacije nad sirovinama. Axum prosleđuje pripremljene instance funkcijama za obradu zahteva preko Extension mehanizma.

Na ovaj način zavisnosti su vidljive na mestu konstrukcije, umesto da ih svaki objekat samostalno stvara tokom obrade. Angular koristi ugrađeni sistem ubrizgavanja kroz inject, deklaracije servisa i pružaoce zavisnosti. Ograničenje serverske realizacije jeste vezanost većine konstruktora za konkretne tipove, pa zamena repozitorijuma testnom implementacijom nije univerzalno omogućena samo prisustvom ovog obrasca. Deljena instanca preko Arc-a ili Angular servisa ne predstavlja sama po sebi implementaciju obrasca Singleton.

5.6.3 DTO i mapiranje podataka

Objekti za prenos podataka, odnosno DTO strukture, definišu podatke koji prelaze granicu interfejsa. U projektu se zahtevi za kreiranje i izmenu razlikuju od modela baze, a složeni odgovori sastavljaju se namenskim funkcijama. ProductionBatchResponse objedinjuje seriju, proizvodne korake i sirovine, dok map_order_response priprema odgovor porudžbine i njenih stavki.

PublicProvenance izdvaja skup informacija dozvoljenih za javni prikaz i generisanje dokumenta. Takvo mapiranje sprečava da se interni model neposredno proglasi javnim ugovorom. Prednost je kontrola sadržaja i mogućnost razvoja šeme baze bez automatskog izlaganja svih promena klijentu. Potrebno je, međutim, održavati doslednost između modela, mapiranja i korisničkog interfejsa, jer tipizovane strukture ne uklanjaju mogućnost semantičke neusaglašenosti.

5.6.4 Strategy i Simple Factory

Strategy izdvaja promenljivo ponašanje iza zajedničkog ugovora. U servisu proizvoda ugovor ProductPolicy definiše metodu validate, dok FoodProductPolicy i GenericProductPolicy predstavljaju konkretne strategije. Zajedničke provere obuhvataju naziv, tip, cenu i količinu. Strategija za prehrambene proizvode dodatno zahteva jedinicu mere.

ProductPolicyFactory::for_type bira strategiju prema tipu proizvoda i vraća objekat koji implementira zajednički ugovor. ProductService primenjuje izabranu strategiju pri izmeni proizvoda, a obrada proizvodnih događaja koristi je pri validaciji izlaza pre upisa. HTTP sloj zato ne sadrži zasebne

grane za sve porodice proizvoda. Dodavanje nove politike može se ostvariti uvođenjem strategije i dopunom mesta izbora.

Realizacija izbora odgovara jednostavnoj fabrici, odnosno Simple Factory pristupu, jer jedna funkcija odlučuje koji objekat da konstruiše. Nije uspostavljena hijerarhija kreatora sa predefinisanim metodama stvaranja karakteristična za GoF Factory Method. Takođe, sva pravila prodaje nisu preneti u strategije: pozitivna prodajna cena, završena serija i zabrana promene potvrđene količine ostaju odgovornost poslovnog servisa. Ovakva podela odgovara stvarnom obimu primene obrasca.

5.6.5 Builder i Director za dokument o poreklu

Builder razdvaja postupak sastavljanja složenog rezultata od njegove konkretne reprezentacije. U Python komponenti za PDF, CertificateBuilder definiše korake product, production, materials i disclaimer, kao i operaciju result. ReportLabCertificateBuilder implementira ove korake kroz paragrafe, tabele, stilove i listu elemenata dokumenta. CertificateDirector određuje redosled izgradnje.

Tok generisanja počinje pribavljanjem javnih podataka o poreklu. Rust servis ih prosleđuje Python procesu, a funkcija render povezuje direktora sa konkretnim graditeljem. Direktor dodaje podatke o proizvodu, proizvodnji, sirovinama i napomenu o izvoru podataka, nakon čega preuzima gotov PDF. Time je pravilo organizacije sadržaja odvojeno od ReportLab detalja.

U projektu postoji RecordingBuilder namenjen proveriti redosleda poziva direktora. Takav test pokazuje da se saradnja učesnika može proveriti bez oslanjanja na vizuelni izgled PDF-a. Drugi graditelj mogao bi podržati drugačiju reprezentaciju uz očuvan redosled koraka, ali takva dodatna reprezentacija nije deo aktuelne implementacije. Obrazac ovde ima jasne učesnike i stvarnu upotrebu u poslovnom toku, nezavisno od bibliotečkih klasa sličnog naziva.

5.6.6 Publish–Subscribe i reaktivne pretplate

Publish–Subscribe razdvaja objavljivanje događaja od pojedinačnih reakcija na njega. Servis proizvodnje objavljuje promenu serije, a RabbitMQ je prosleđuje odgovarajućim redovima. Potrošač projekcija osvežava podatke za čitanje, dok potrošač izlaza proizvodnje ažurira proizvode. Proizvođač događaja ne poziva svaki od ovih tokova zasebno, a potrošači obrađuju poruke nezavisno.

Na korisničkoj strani, RxJS Observable i pretplate omogućavaju reakciju na odgovore i vremenske događaje [19]. Primer je periodično osvežavanje upravljačkog pregleda uz prekid pretplate pri zatvaranju komponente. Ovakva primena ima vezu sa principom Observer obrasca, ali distribuirana komunikacija preko RabbitMQ-a nije klasična implementacija objekta koji u istoj memoriji održava listu posmatrača. Razlikovanje ova dva nivoa važno je zbog različitih uslova otkaza i trajnosti poruka.

5.6.7 Transactional Outbox

Transactional Outbox rešava problem nepouzdanog odvojenog upisa poslovne promene i objavljivanja događaja [27]. Ukoliko se baza uspešno izmeni, a broker u tom trenutku nije dostupan, događaj ne sme trajno nestati. U LocalBite-u okidači upisuju zapis u `integration_outbox` u istoj lokalnoj transakciji sa promenom poslovnog objekta. Pozadinski proces zatim čita neposlate zapise, objavljuje ih i označava kao poslate nakon potvrde brokera.

Primer je završavanje serije: status i događaj ostaju sačuvani zajedno čak i kada RabbitMQ privremeno nije dostupan. Nakon oporavka, događaj može stići do servisa proizvoda i projekcija. Ako dođe do prekida nakon objavljivanja, a pre označavanja zapisa, poruka može biti poslata ponovo. Obrazac zato zahteva idempotentne potrošače, kao i održavanje evidencije događaja i praćenje zaostale obrade. Njegova garancija odnosi se na vezu lokalne promene i zapisa događaja, a ne na jednu transakciju koja obuhvata sve servise.

5.6.8 Idempotent Consumer i zaštita od zastarelih događaja

Potrošač projekcija beleži obrađene poruke u `projection_receipts` prema izvoru i sekvenci događaja. Zapis o prijemu i promena projekcije izvršavaju se u istoj transakciji. Ako ista poruka ponovo stigne, postojeći zapis sprečava ponovnu primenu. Dodatno, sekvenca na projektovanom entitetu obezbeđuje da stariji događaj ne prepíše novije stanje.

Logička oznaka brisanja zadržava informaciju da je entitet uklonjen, kako ranije poslata poruka ne bi ponovo uspostavila njegov prikaz. U servisu proizvoda odvojeni mehanizam koristi stanje proizvodnje, identitete izlaza i evidenciju `production_outputs`. Time se ponavljanje promene serije ne tumači kao nalog za stvaranje nove zalihe. Ovakve zaštite odnose se na konkretne operacije; ne znače da su svi pozivi aplikacije automatski idempotentni.

5.6.9 CQRS i projekcije

CQRS razdvaja odgovornosti modela za menjanje podataka i modela za njihovo čitanje [28]. Domenski servisi LocalBite-a proveravaju poslovna pravila i čuvaju merodavno stanje. Servis `Read Models` iz događaja formira projekcije pogodne za upravljački pregled i poreklo proizvoda. Čitanje porekla zato ne zahteva da svaki javni zahtev posebno pribavlja sve podatke iz servisa korisnika, proizvodnje i sirovina.

Prednost je mogućnost oblikovanja prikaza prema potrebama čitanja, uz očuvanje vlasništva nad izvornim podacima. Ograničenje je vremenski razmak između promene izvora i osvežavanja projekcije.

Primena je selektivna, jer pojedini pregledi i analitika ostaju u domenskim servisima. Projekcije su aplikacione tabele i nisu PostgreSQL materijalizovani pogledi.

Sistem ne primenjuje potpuni Event Sourcing: trenutno poslovno stanje čuva se u redovnim tabelama, a integracioni događaji služe prenosu promena i obnovi projekcija. Mogućnost ponovnog slanja outbox događaja nije dovoljna da bi se celo rešenje označilo kao sistem zasnovan na rekonstrukciji stanja isključivo iz dnevnika događaja.

5.6.10 Trajna orkestracija i kompenzacioni tokovi

Saga pristup koordinira više lokalnih transakcija i predviđa kompenzacije kada se poslovni tok ne može završiti [29]. LocalBite koristi trajno evidentirane korake i kompenzacije operacije koji slede ovaj princip, bez posebnog opšteg izvršnog okvira za sage. Servis porudžbina čuva nameru u `checkout_jobs`, zahteva rezervaciju proizvoda i zatim upisuje porudžbine i sačuvani odgovor. Pozadinski proces nastavlja obradu nezavršenih zahteva istim identifikatorom.

Ako je rezervacija već izvršena pre prekida procesa, njeno ponavljanje ne sme ponovo umanjiti zalihi. Kada vlasnik otkaze porudžbinu, zajedno sa promenom statusa upisuje se `stock_release_jobs` zapis. Oslobođanje rezervacije zatim se ponavlja do uspešnog izvršavanja, uz idempotentnu obradu u servisu proizvoda. Zbog toga povraćaj ne mora biti vidljiv u istom trenutku kada je prihvaćeno otkazivanje.

Sličan mehanizam postoji za sirovine: `material_intents` čuva nameru potrošnje, dok `material_release_jobs` evidentira potreban povraćaj. Okidači kreiraju poslove povraćaja pri dozvoljenom uklanjanju veze sirovine, otkazivanju ili logičkom brisanju serije. Oporavak razlikuje uspešno povezanu sirovinu od prekinute operacije i šalje odgovarajući zahtev servisu zaliha. Kompenzacija predstavlja novu poslovnu operaciju kojom se ispravlja posledica prethodnog koraka; ne predstavlja vraćanje jedne distribuirane SQL transakcije.

5.6.11 Konačne mašine stanja

Životni ciklusi porudžbine, serije i proizvodnog koraka predstavljeni su eksplicitnim skupovima stanja i dozvoljenih prelaza. `OrderStatus` definiše dozvoljene naredne statuse, dok `BatchService` i `StepService` proveravaju promene stanja proizvodnje. Okidači baze dodatno štite vezu između statusa koraka i zatvaranja serije, uključujući konkurentne zahteve.

Ovakav pristup čini poslovne prelaze preglednim: porudžbina ne može neposredno preći iz čekanja u isporučeno stanje, a serija se ne može završiti dok postoje nezavršeni koraci. Međutim, u kodu ne postoji poseban objekat sa ponašanjem za svako stanje, što bi karakterisalo objektnu realizaciju GoF State

obrasca. Preciznije je govoriti o konačnim mašinama stanja, realizovanim enumeracijama, uslovima i ograničenjima baze.

5.6.12 Istorijski snimci i logičko brisanje

Istorijski snimci izdvajaju podatke relevantne za prethodno izvršenu operaciju. Veza sirovine i serije čuva podatke zabeležene pri utrošku, a stavka porudžbine naziv, cenu i jedinicu proizvoda u trenutku kupovine. Kasnija promena dobavljača ili prodajne cene zato ne menja automatski značenje ranije evidencije. Kod izlaza proizvodnje zadržava se i prethodno sačuvan plan, kada postoji, uz potvrđene vrednosti.

Logičko brisanje koristi oznaku `is_deleted` za izdvajanje zapisa iz redovnih pregleda uz očuvanje podataka potrebnih za istoriju i integraciju. Ovaj pristup zahteva dosledne filtere u svim relevantnim upitima i nije podjednako primenjen na sve tabele. Istorijski snimci i logičko brisanje predstavljaju obrasce rada sa podacima, a ne samostalnu realizaciju nepromenljivog revizionog dnevnika.

5.6.13 Zajednički doprinos obrazaca

Primenjeni obrasci rešavaju različite delove istog poslovnog problema. Repository, Service Layer i ubrizgavanje zavisnosti određuju organizaciju koda; Strategy izdvaja pravila validacije, a Builder postupak sastavljanja dokumenta. Outbox, nezavisni potrošači, projekcije i kompenzacije povezuju servise tako da privremeni prekid komunikacije ne mora dovesti do trajnog gubitka poslovnog toka.

Njihova vrednost procenjuje se kroz konkretno ponašanje: završena serija mora sačuvati potvrđene izlaze, ponovljena poruka ne sme duplirati zalihu, a otkazana porudžbina mora pokrenuti oslobađanje rezervacije. Primena obrazaca ujedno povećava broj komponenti i evidencija koje treba održavati. Zbog toga je svaka apstrakcija u radu vezana za postojeći problem implementacije i njena ograničenja, umesto da se broj imenovanih obrazaca posmatra kao samostalan pokazatelj kvaliteta.

6 Model podataka i servisne baze

6.1 Tri nivoa modela

Model podataka može se posmatrati na tri nivoa. Domen opisuje poslovne pojmove kao što su sirovina, serija i proizvod. Persistence model opisuje tabele, kolone, indekse i trigere. DTO model opisuje strukturu javnog ili međuservisnog ugovora. Razdvajanje sprečava da interna struktura baze postane automatski javni API.

6.2 Auth baza

Tabela users čuva identitet, ulogu, kontakt i farm_id. Lozinka se ne čuva u izvornom obliku, već kao Argon2 hash. Tabela farms čuva naziv, adresu, kontakt, opis i sajt. Kreiranje preduzeća i dodela vlasnika odvijaju se u lokalnoj transakciji, uz zaštitu od drugog paralelnog preduzeća za istog vlasnika.

6.3 Raw Materials baza

Sirovina ima naziv, tip, količinu, jedinicu mere, prag niske zalihe i opcione datume porekla. Potrošnja se zapisuje u ledger, a operacija ima identifikator koji podržava bezbedno ponavljanje. Jedinica potrošnje mora tekstualno odgovarati jedinici sirovine; sistem ne radi automatsku konverziju između kilograma i grama ili litara i mililitara.

6.4 Productions baza

Seriya proizvodnje ima status, naziv procesa, početak, kraj i opseg podataka preduzeća. Koraci su uređeni i sadrže opis, trajanje i temperaturu. Veza sa sirovinom čuva istorijski snapshot naziva, tipa, jedinice, porekla i količine. Po završetku serija nosi podatke o izlazu: naziv, tip, jedinicu i pozitivnu količinu.

6.5 Products i Orders baze

Products tabela predstavlja proizvod u skladištu i ponudi. Neaktivan proizvod može nastati iz završene serije, dok aktiviranje zahteva pozitivnu cenu, važeću seriju i raspoloživu količinu. Orders čuva porudžbinu, stavke, snapshot cene i naziva, status i veze ka rezervaciji. Istorijski snapshot štiti račun od kasnije promene cene u katalogu.

Baza	Glavne tabele	Posebna pravila
auth_db	users, farms	role i pripadnost preduzeću
raw_materials_db	raw_materials, ledger	atomska potrošnja
productions_db	batches, steps, usages	statusna nepromenljivost
products_db	products, media, outputs	stock reservation i QR
orders_db	orders, items, checkout_jobs	idempotency i release
read_models_db	projections, receipts	sequence i tombstone

Tabela 7: Pregled servisnih baza

7 Implementacija backend servisa

7.1 Auth servis

Auth servis obrađuje registraciju, prijavu, profil, preduzeće i zaposlene. Javna registracija prihvata samo CUSTOMER i FARM_OWNER. Radnika kreira vlasnik kroz zaštićeni tok, a SYSTEM_ADMIN se ne registruje javno. Nakon kreiranja preduzeća izdaje se novi JWT sa farm_id, čime se naredni zahtevi mogu objektno ograničiti.

7.2 Raw Materials servis

Servis sirovina centralizuje proveru količine i jedinice. Potrošnja se obavlja atomski, tako da dva konkurentna zahteva ne mogu oba uspešno da potroše istu poslednju količinu. Za uklanjanje veze sa serijom koristi se kompenzacioni posao, jer je udaljena izmena u drugoj bazi neizbežno odvojena od lokalnog commita.

7.3 Productions servis

Productions upravlja životnim ciklusom serije. PLANNED omogućava pripremu, IN_PROGRESS označava aktivnu obradu, a COMPLETED zaključava osnovne podatke i emituje izlaz. CANCELLED predstavlja odustajanje. Završene i otkazane serije ne prihvataju izmene kroz regularni update, dok su koraci zaštićeni dodatnim pravilima i triggerima.

7.4 Products servis

Products je granica između proizvodnje i prodaje. Consumer događaja kreira proizvod u skladištu tek nakon završene serije. Poseban zapis production_outputs i zaključavanje batch-a sprečavaju dvostruko kreiranje ako broker ponovi istu poruku. Aktiviranje proizvoda proverava cenu, status serije, opseg podataka preduzeća i rok trajanja.

7.5 Orders servis

Orders prihvata listu stavki, proverava identitet kupca i preko Products servisa dobija trenutne cene i zalihe. Stavke se grupišu po preduzeću i formiraju pojedinačne porudžbine. Pre lokalnog upisa kreira se checkout intent, a nakon rezervacije zalihe upisuju se porudžbine i stavke. Ako servis padne između koraka, recovery ponavlja identičnu operaciju.

7.6 Zajednička biblioteka

Common crate ujednačava JWT strukture, greške, envelope događaja, identifikatore i pomoćne tipove. Zajednički kod smanjuje razlike između servisa, ali povećava razvojnu povezanost: promena ugovora mora biti kompatibilna sa svim potrošačima i publisher-ima. Zbog toga `schema_version` i plan evolucije predstavljaju sastavni deo integracije.

8 Integracija servisa i distribuirani tokovi

8.1 Transakcioni outbox

Kod svake poslovne izmene najvažniji zahtev je da zapis i događaj ne budu razdvojeni. Ako se baza upiše, a proces padne pre objavljivanja poruke, druga baza neće saznati za promenu. Ako se poruka objavi pre commit-a, može opisivati promenu koja na kraju nije sačuvana. Transakcioni outbox rešava prvi deo problema tako što događaj čuva u istoj lokalnoj transakciji kao poslovni zapis.

Relay periodično čita neobjavljene redove, zaključava rad sa advisory lock-om i šalje persistent poruke uz publisher confirm. Ako potvrda ili upis published_at izostane, poruka može biti poslata ponovo. Zato potrošač ne sme da pretpostavi da će događaj stići tačno jednom.

8.2 Projekcije, receipts i tombstones

Read-model consumer obrađuje envelope, proverava izvor i sekvencu i u istoj lokalnoj transakciji upisuje receipt i projekciju. Receipt sa jedinstvenim parom source i sequence sprečava ponovnu primenu. Uslovni upsert prihvata samo noviju sekvencu. Tombstone sprečava da stari INSERT posle brisanja oživi entitet.

8.3 Tok proizvodnog izlaza

Kada Productions objavi završenu seriju, Products consumer proverava envelope i output. Ako output još ne postoji, nastaje neaktivan proizvod i zapis production_outputs. Ako se poruka ponovi, zaključavanje i jedinstveni zapis vraćaju postojeći rezultat. Time je nastanak zalihe idempotentan i odvojen od trenutka kada je korisnički interfejs dobio odgovor o završetku serije.

8.4 Kompenzacija potrošnje

Sirovinska potrošnja je raspodeljena između Productions i Raw Materials. Productions prvo upisuje material_intent, zatim poziva udaljeni servis sa idempotentnim ključem. Ako se veza uklanja ili serija otkazuje, stvara se material_release_job. Recovery ponavlja release dok zavisni servis ne bude dostupan. Ovo nije globalni rollback, već trajna eventualna kompenzacija.

8.5 Porudžbina i rezervacija

Tok porudžbine koristi atomsku rezervaciju svih stavki. Products zaključava proizvode, proverava cenu, aktivnost, rok i količinu, a zatim umanjuje stanje. Orders lokalno čuva rezultat i veze rezervacije. U slučaju pada nakon rezervacije, checkout_jobs ostaje PENDING i recovery ponavlja zahtev sa istim ključem. Otkazivanje stvara stock_release_job koji vraća rezervaciju.

Kvar	Očekivano ponašanje	Granica
Broker nedostupan	outbox čeka objavljivanje	projekcija kasni
Dupli događaj	receipt odbija ponavljanje	važi za pokrivene tokove
Stariji događaj	sequence uslov ne menja noviji red	nema globalnog redosleda
Orders padne	checkout intent omogućava recovery	potrebno nadzirati poslove
Read baza nedostupna	consumer requeue/reconnect	upiti privremeno ne uspevaju
Loša poruka	dead-letter ili potvrđeni DLQ	zahteva operativni pregled

Tabela 8: Matrica kvarova

9 Frontend i korisnički interfejs

9.1 Organizacija Angular aplikacije

Frontend je organizovan po funkcionalnim celinama. Javna stranica porekla i PDF pristup izdvojeni su od zaštićenih ekrana. Sesija se zasniva na tokenu i profilu korisnika, dok guards i servisni wrapper-i određuju koje rute i komande su dostupne. TypeScript modeli drže ugovor sa backendom eksplicitnim i smanjuju greške pri promeni naziva polja.

9.2 Ekрани i tokovi

Vlasnik i radnik koriste dashboard, pregled sirovina, proizvodne serije, korake, skladište i ponudu. Kupac koristi katalog, filter proizvođača, korpu i porudžbine. Javna stranica porekla prikazuje proizvod, preduzeće, seriju, korake i istorijske snimke sirovina. Analitika prikazuje prihod isporučenih porudžbina, broj porudžbina i raspodelu statusa.

9.3 Forme i validacija

Frontend validacija daje brzu povratnu informaciju, ali backend ostaje autoritet. Količine se prikazuju sa ograničenom preciznošću, datumi se proveravaju prema pravilima domena, a poruke grešaka povezuju se sa konkretnim poljima. Kod distribuiranih tokova korisnik dobija informaciju da se proizvod ili poreklo možda još projektuju, umesto da se asinhronost prikrije kao greška aplikacije.

9.4 Sesija i dozvole

Dozvole su kombinacija uloge i konteksta preduzeća. Frontend može sakriti dugme, ali to nije bezbednosna granica. Backend ponovo proverava JWT, opseg podataka preduzeća, vlasništvo i efektivnu aktivnost objekta. Tokeni za tehničke pozive kratkog su trajanja i vezani za operaciju ili resurs.

9.5 Javni QR prikaz

QR kod ne sadrži kompletne poslovne podatke, već URL sa tokenom. Javni DTO je allowlist struktura: ne izlaže interne identifikatore, cenu, stanje, dobavljača, putanje ni nepotrebne datume. Products proverava da proizvod postoji, da je aktivan i da nije logički obrisan. Read-model zatim dodaje projekcije porekla.

9.6 Pristupačnost i povratne informacije

Forme koriste labele, poruke grešaka i osnovne ARIA attribute. Za složeniju proveru bila bi potrebna formalna WCAG procena i testiranje čitačem ekrana. To je naročito važno za javni QR prikaz, jer korisnik može doći sa mobilnog uređaja i bez prethodnog poznavanja platforme.

10 Bezbednost, pouzdanost i ograničenja

10.1 Autentifikacija i autorizacija

Lozinke se čuvaju Argon2 heširane. Korisnički JWT traje jedan sat, dok tehnički tokeni traju približno minut i vezani su za operaciju, resurs i preduzeće. Uloge se proveravaju na ruti, a objekti se dodatno proveravaju prema farm_id. Ovakva dvostruka provera sprečava da korisnik sa ispravnom ulogom pristupi objektu drugog preduzeća.

10.2 Zaštita zalihe

Javni decrement je uklonjen. Zaliha se menja preko interne atomske rezervacije sa idempotency ključem. Products u jednoj transakciji zaključava sve stavke, proverava uslove i umanjuje količine. Time se smanjuje rizik da dva kupca uspešno kupe poslednju jedinicu.

10.3 Privatnost javnog prikaza

Javni prikaz mora da bude informativan, ali minimalan. Interni ID-evi, kontakt podaci, cena, stanje, dobavljač i filesystem putanje nisu deo PublicProvenance DTO-a. Logovi izbegavaju tokene, payload i stderr procesa za PDF. Mediji se preuzimaju autorizovanim HTTP zahtevom, a direktan javni /uploads je uklonjen.

10.4 Preostale granice poverenja

JWT je stateless, pa promena uloge ili članstva ne opoziva već izdat token pre isteka. Token u localStorage ostavlja XSS kao važnu browser pretnju. Servisi dele simetrični JWT secret, CORS je široko podešen za razvoj, a RabbitMQ je jedna instanca. Nema formalnog penetration testa ni centralnog distributed tracing-a.

10.5 Tehnički dug

ID	Ograničenje	Predložena dopuna
O-01	zamena slike može ostaviti delimičan ishod	write-new, DB swap, delete-old
O-02	WebP je naveden ali nije potpuno pokriven	uskladiti allowlist i testove
O-03	null često znači bez promene	uvesti tri-state update model
O-04	liste učitavaju stavke po porudžbini	batch query ili agregacija
O-05	nema centralnih metrika	metrike, correlation ID i tracing
O-06	uploads volume nije deljen	objektni storage
O-07	retention nije automatski	odobriti rokove i dry-run posao

Tabela 9: Tehnički dug i predložene dopune

11 Testiranje i verifikacija

11.1 Strategija testiranja

Testiranje je organizovano po slojevima. Jedinični i servisni testovi proveravaju pravila, dok integracioni testovi pokreću baze i servise u Docker okruženju. Posebne skripte proveravaju tok poslovanja i porekla, konkurentnu rezervaciju, CQRS, output događaje, backup i frontend build. Cilj nije samo potvrda uspešnog slučaja, već proveru ponašanja posle prekida i dupliranja.

11.2 Rezultati provera

Provera	Rezultat	Šta je potvrđeno
Rust cargo test release workspace	PASS	backend testovi i kompajliranje
Certificate test	2 PASS	Builder, Director i PDF izlaz
Business trace verification	PASS	scope, uloge, stock i poreklo
CQRS verification	PASS	publish i projekcije
Output events verification	PASS	stariji događaj ne oživljava zalihi
Backup verify	PASS, 6/6	restore svih baza
Angular production build	PASS	frontend tipovi i build
HTTP baseline	500/500	lokalni zahtevi bez greške

Tabela 10: Rezultati verifikacije

11.3 Testiranje autorizacije i scope-a

Provere obuhvataju pokušaj spoof-ovanja identiteta, javnu registraciju nedozvoljenih uloga, pristup tuđoj porudžbini, pristup tuđem proizvodu i promenu objekta drugog preduzeća. Ovi testovi su važniji od same provere statusnog koda, jer potvrđuju da opseg podataka preduzeća prolazi kroz sve relevantne slojeve.

11.4 Testiranje distribuiranih tokova

CQRS test zaustavlja RabbitMQ i source servise, čime se proverava da se lokalni upis ne gubi i da se projekcija kasnije može nadoknaditi. Output test šalje starije ili duplirane događaje i proverava sequence zaštitu. Order test simulira pad Orders-a posle rezervacije i proverava da checkout recovery završava ili bezbedno ponavlja operaciju.

11.5 Performanse

Lokalni baseline izvršava 100 read-only zahteva sa konkurentnošću 5, ukupno 500 zahteva po endpointu. Rezultati za health rute su reda nekoliko milisekundi u zagrejanom Docker loopback okruženju. Ovaj rezultat nije produkcion SLA: ne meri checkout, PDF, broker backlog, mrežu, opterećenje baza ni p95/p99 za poslovne upite.

11.6 Ograničenja dokaza

Uspešan lokalni test ne dokazuje ponašanje pod višestrukim instancama svih servisa, mrežnim particijama, gubitkom storage-a ili kvarom hosta. Za produkciju su potrebni chaos testovi, testovi kapaciteta, formalna bezbednosna analiza, periodični restore drill i merenje starosti projekcija.

12 Pokretanje, održavanje i oporavak

12.1 Pokretanje sistema

Osnovno lokalno pokretanje obavlja se kroz Docker Compose. Nginx je ulazna tačka, a servisi se pokreću sa konfiguracijom za lokalni DNS i baze. Migracije se primenjuju u okviru servisnih procesa ili pripremljenih skripti. Razvojni portovi ne treba da budu direktno izloženi u produkciji.

```
docker compose up -d nginx
node scripts/verify-role-stock.cjs
node scripts/verify-farm-trace.cjs
node scripts/verify-cqrs.cjs
node scripts/verify-output-events.cjs
node scripts/backup-verify.cjs
```

12.2 Health i monitoring

Health endpoint potvrđuje liveness procesa, ali ne garantuje da su outbox redovi prazni, da su sve projekcije ažurne ili da su zavisnosti dostupne. Operativno treba pratiti starost najstarijeg outbox reda, dubinu RabbitMQ redova i DLQ-a, broj pending checkout, material i stock-release poslova, stanje consumer-a i vreme poslednje projekcije.

12.3 Backup i restore

Autoritativni skup čine pet write baza, read_models baza i uploads. Backup skripta pravi dump svake baze, arhivu uploads-a i manifest sa SHA-256 vrednostima. Svaki dump se vraća u izolovanu PostgreSQL 15 instancu. Read model može biti vraćen iz backup-a ili rekonstruisan replay-em outbox događaja.

12.4 Koordinisana obnova

- zaustaviti write saobraćaj i zabeležiti checkpoint
- sačuvati write baze i uploads u istom maintenance prozoru
- proveriti manifest i kontrolne sume
- vratiti baze i uploads
- vratiti read model ili pokrenuti replay do praznog backlog-a
- proveriti health, DLQ, opseg podataka preduzeća, javni trag i checkout smoke test
- otvoriti write saobraćaj nakon verifikacije

12.5 Retention

Neobjavljeni outbox se ne briše. Objavljeni outbox može imati početni rok od 90 dana samo ako backup i replay politika to dozvoljavaju. Receipts i tombstones moraju ostati najmanje koliko i replay istorija, jer bi njihovo rano brisanje omogućilo ponovnu primenu starog događaja. Poslovne porudžbine podležu pravnim zahtevima, a ne samo tehničkom cleanup-u.

13 Diskusija rezultata i budući razvoj

13.1 Ostvareni cilj

LocalBite ostvaruje osnovni cilj povezivanja proizvodnje, zalihe, prodaje i javnog porekla. Najvažnija vrednost nije pojedinačna CRUD funkcija, već kontinuitet identiteta: sirovina dobija vezu sa serijom, serija proizvodi zalihi, zaliha postaje ponuda, a porudžbina čuva istorijsku cenu i količinu. QR prikaz taj lanac prevodi u oblik razumljiv kupcu.

Arhitektura je dovoljno eksplicitna da se mogu razlikovati autoritativni zapisi i projekcije. Lokalna transakcija štiti svaku domensku bazu, dok outbox i receipts štite integraciju u pokrivenim scenarijima. Kompenzacioni poslovi priznaju činjenicu da udaljena operacija ne može biti vraćena običnim rollback-om lokalne baze.

13.2 Trade-off mikroservisa

Izbor mikroservisa opravdan je različitim domenima i potrebom za jasnim vlasništvom. Ipak, aplikacija nije potpuno nezavisna: servisi dele common crate, JWT secret, Compose konfiguraciju i deo razvojnih procedura. To nije greška, već realna cena koherentnog proizvoda. Arhitektura treba da ostane dovoljno jednostavna da operativni tim može da razume tok kvarova.

13.3 Prioriteti naredne iteracije

Prvi prioritet je operativna vidljivost: centralne metrike, correlation ID, tracing i alarmi za najstarije poslove. Drugi je pouzdaniji medijski tok sa write-new i bezbednom zamenom datoteke. Treći je produkcionni load test sa dogovorenim SLO, RPO i RTO vrednostima. Tek nakon toga ima smisla uvoditi elektronsko plaćanje ili AI prognoziranje potražnje.

13.4 Moguće proširenje domena

Buduća verzija može dodati normativ proizvodnje, očekivani prinos, automatski otpis i konverziju jedinica. Mobilni klijent i offline sinhronizacija zahtevaju dodatni konfliktni model. Nezavisna sertifikacija zahteva izvor izvan unosa proizvođača, na primer kontrolu kvaliteta ili potpis ovlašćenog lica. Svako proširenje treba da očuva javni DTO i opseg podataka preduzeća.

13.5 Naučene lekcije

- servisna granica mora pratiti poslovno vlasništvo, a ne samo tehnički folder
- eventualna konzistentnost mora biti vidljiva u korisničkom iskustvu
- idempotentnost je obavezna za sve operacije koje mogu biti ponovljene

- javni prikaz treba da bude allowlist, a ne kopija interne tabele
- testovi kvarova otkrivaju probleme koje happy path ne pokazuje

14 Zaključak

U radu je prikazana platforma LocalBite za praćenje proizvodnje i porekla domaćih proizvoda. Polazni problem rasute evidencije rešen je objedinjavanjem sirovina, proizvodnih serija, zaliha, ponude, porudžbina i javnog QR prikaza. Model preduzeća kao poslovne granice omogućava više korisnika u jednoj aplikaciji uz kontrolu pristupa i izolaciju podataka.

Implementirano rešenje koristi mikroservisnu arhitekturu sa zasebnim PostgreSQL bazama, REST pozivima i RabbitMQ događajima. Transakcioni outbox obezbeđuje da se poslovna promena i događaj sačuvaju zajedno u lokalnoj bazi. Projection receipts, sequence provere i tombstones omogućavaju bezbednu obradu duplikata i starijih događaja. Kod zaliha, idempotentna atomska rezervacija i trajni checkout intent smanjuju rizik dvostruke prodaje ili gubitka operacije posle prekida.

Analiza je pokazala da sistem ima jasne prednosti, ali i precizno definisane granice. Projekcije su eventualno konzistentne, nema globalne distribuirane transakcije, RabbitMQ lokalno nije visoko dostupan, a javni PDF prikazuje podatke koje unosi proizvođač. Ograničenja nisu skrivena od korisnika rada; ona predstavljaju osnovu za planiranje naredne iteracije i pravilno tumačenje rezultata.

Verifikacija u lokalnom Docker okruženju potvrdila je build, servisne testove, tokove autorizacije, CQRS, output događaje, backup i osnovni performance baseline. Time je ostvaren cilj rada: projektovati i dokumentovati funkcionalnu platformu čija arhitektura podržava sledljivost proizvodnje, a čiji su mehanizmi pouzdanosti i preostali rizici jasno obrazloženi.

Doprinos rada

- formalizovan tok od sirovine do javnog porekla
- odvojeno vlasništvo nad podacima po servisima
- implementirana zaštita od duplog trošenja i duplog output-a
- objašnjena eventualna konzistentnost kroz konkretne tokove
- verifikacija uspešnih i neuspešnih distribuiranih scenarija
- operativni plan za backup, replay, retention i budući razvoj

Literatura

- [1] N. Zorić, „LocalBite - Mikroservisna platforma za praćenje proizvodnje i porekla domaćih proizvoda“, stručni rad, Fakultet tehničkih nauka, Univerzitet u Novom Sadu.
- [2] N. Zorić, „Projektna dokumentacija: Local Bite - detaljna specifikacija sistema“, interna tehnička dokumentacija projekta, 2026.
- [3] D. Božović, „TempleOS: architecture and principles of lightweight operating system development“, diplomski rad, Fakultet tehničkih nauka u Čačku, Univerzitet u Kragujevcu, 2024.
- [4] S. Newman, Building Microservices, 2nd ed., O'Reilly Media, 2021.
- [5] J. Lewis i M. Fowler, „Microservices“, 2014. Dostupno na: <https://martinfowler.com/articles/microservices.html> (pristupljeno 24. 9. 2026).
- [6] R. T. Fielding, Architectural Styles and the Design of Network-based Software Architectures, PhD thesis, University of California, Irvine, 2000.
- [7] M. Jones, J. Bradley and N. Sakimura, JSON Web Token (JWT), RFC 7519, IETF, 2015.
- [8] PostgreSQL Global Development Group, „PostgreSQL 15 Documentation — What Is PostgreSQL?“. Dostupno na: <https://www.postgresql.org/docs/15/intro-what-is.html> (pristupljeno 24. 9. 2026).
- [9] RabbitMQ, „Consumer Acknowledgements and Publisher Confirms“, dokumentacija. Dostupno na: <https://www.rabbitmq.com/docs/confirms> (pristupljeno 24. 9. 2026).
- [10] Rust Project Developers, „The Rust Programming Language — Understanding Ownership“. Dostupno na: <https://doc.rust-lang.org/book/ch04-00-understanding-ownership.html> (pristupljeno 24. 9. 2026).
- [11] Angular Team, „What is Angular?“, zvanična dokumentacija. Dostupno na: <https://angular.dev/overview> (pristupljeno 24. 9. 2026).
- [12] OWASP Foundation, Application Security Verification Standard and Authentication Cheat Sheet.
- [13] Axum contributors, „axum 0.8.8“, dokumentacija biblioteke. Dostupno na: <https://docs.rs/axum/0.8.8/axum/> (pristupljeno 24. 9. 2026).

- [14] Tokio contributors, „Tutorial“, zvanična dokumentacija. Dostupno na: <https://tokio.rs/tokio/tutorial> (pristupljeno 24. 9. 2026).
- [15] SQLx contributors, „sqlx 0.8.6“, dokumentacija biblioteke. Dostupno na: <https://docs.rs/sqlx/0.8.6/sqlx/> (pristupljeno 24. 9. 2026).
- [16] Serde contributors, „Overview“, zvanična dokumentacija. Dostupno na: <https://serde.rs/> (pristupljeno 24. 9. 2026).
- [17] A. Biryukov, D. Dinu, D. Khovratovich i S. Josefsson, „Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications“, RFC 9106, 2021. Dostupno na: <https://www.rfc-editor.org/rfc/rfc9106.html> (pristupljeno 24. 9. 2026).
- [18] Microsoft, „The TypeScript Handbook“, zvanična dokumentacija. Dostupno na: <https://www.typescriptlang.org/docs/handbook/intro.html> (pristupljeno 24. 9. 2026).
- [19] ReactiveX, „RxJS Introduction“, zvanična dokumentacija. Dostupno na: <https://rxjs.dev/guide/overview> (pristupljeno 24. 9. 2026).
- [20] ReportLab, „Chapter 5: Platypus“, ReportLab User Guide. Dostupno na: https://docs.reportlab.com/reportlab/userguide/ch5_platypus/ (pristupljeno 24. 9. 2026).
- [21] Docker Inc., „Docker Compose“, zvanična dokumentacija. Dostupno na: <https://docs.docker.com/compose/> (pristupljeno 24. 9. 2026).
- [22] NGINX, „Module ngx_http_proxy_module“, zvanična dokumentacija. Dostupno na: https://nginx.org/en/docs/http/ngx_http_proxy_module.html (pristupljeno 24. 9. 2026).
- [23] E. Gamma, R. Helm, R. Johnson i J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994, ISBN 978-0-201-63361-0. Dostupno na: <https://www.informit.com/store/design-patterns-elements-of-reusable-object-oriented-9780201633610> (pristupljeno 24. 9. 2026).
- [24] E. Hieatt i R. Mee, „Repository“, katalog Patterns of Enterprise Application Architecture, 2003. Dostupno na: <https://martinfowler.com/eaCatalog/repository.html> (pristupljeno 24. 9. 2026).
- [25] R. Stafford, „Service Layer“, katalog Patterns of Enterprise Application Architecture, 2003. Dostupno na: <https://martinfowler.com/eaCatalog/serviceLayer.html> (pristupljeno 24. 9. 2026).
- [26] M. Fowler, „Inversion of Control Containers and the Dependency Injection pattern“, 2004. Dostupno na: <https://martinfowler.com/articles/injection.html> (pristupljeno 24. 9. 2026).

[27] C. Richardson, „Pattern: Transactional outbox“, Microservices.io. Dostupno na: <https://microservices.io/patterns/data/transactional-outbox.html> (pristupljeno 24. 9. 2026).

[28] M. Fowler, „CQRS“, 2011. Dostupno na: <https://martinfowler.com/bliki/CQRS.html> (pristupljeno 24. 9. 2026).

[29] C. Richardson, „Pattern: Saga“, Microservices.io. Dostupno na: <https://microservices.io/patterns/data/saga.html> (pristupljeno 24. 9. 2026).

Prilog A API primeri

A.1 Registracija

```
POST /api/auth/register

{
  "email": "kupac@example.com",
  "password": "...",
  "role": "CUSTOMER",
  "first_name": "Ana",
  "last_name": "Jovanović"
}
```

A.2 Završetak proizvodne serije

```
PUT /api/productions/batches/{batch_id}/complete

{
  "output_name": "Domaći med",
  "output_type": "honey",
  "output_unit": "kg",
  "output_quantity": 25.0,
  "expiry_date": "2027-09-18"
}
```

A.3 Javni prikaz porekla

```
GET /api/public/trace/{qr_token}

200 OK

{
  "product": {"name": "Domaći med", "unit": "kg"},
  "farm": {"name": "Primer preduzeća"},
  "batch": {"name": "Prolećna proizvodnja"},
  "steps": []
}
```

Primeri služe za prikaz granice ugovora. Stvarna polja, statusni kodovi i autorizacioni zahtevi određeni su servisnim DTO modelima. Javni DTO treba održavati kao eksplicitnu allowlist strukturu, bez automatskog serijalizovanja interne persistence strukture.

Prilog B Statusi i matrica prava

B.1 Statusi proizvodne serije

Status	Dozvoljeni prelazi	Značenje
PLANNED	IN PROGRESS, CANCELLED	serija je pripremljena
IN PROGRESS	COMPLETED, CANCELLED	obrađa je u toku
COMPLETED	nema	proizvodni rezultat je zaključen
CANCELLED	nema	serija je otkazana

B.2 Statusi porudžbine

Status	Dozvoljeni prelazi	Značenje
PENDING	CONFIRMED, CANCELLED	kreirana, čeka potvrdu
CONFIRMED	SHIPPED, CANCELLED	potvrđena
SHIPPED	DELIVERED	poslata
DELIVERED	nema	isporučena
CANCELLED	nema	otkazana

B.3 Matrica prava

Akcija	Kupac	Radnik	Vlasnik	Posetilac
Pregled javnog porekla	da	da	da	da
Pregled kataloga	da	da	da	ne
Izmena sirovine	ne	da	da	ne
Upravljanje radnicima	ne	ne	da	ne
Završetak serije	ne	da	da	ne
Aktiviranje prodaje	ne	da	da	ne
Kreiranje porudžbine	da	ne	ne	ne

Matrica je sažetak poslovnih pravila, a ne zamena za objektne provjere. Zaštićena ruta uvek mora proveriti korisnika, ulogu, opseg podataka preduzeća i stanje konkretnog objekta.